



UNIVERSIDADE ESTADUAL PAULISTA
“JÚLIO DE MESQUITA FILHO”
Campus de São José do Rio Preto

REDES DE COMPUTADORES

Redes de Computadores (2375SCC) – 2025

Prof. Dr. Adriano Mauro Cansian

adriano.cansian@unesp.br

[linkedin.com/in/adrianocansian/](https://www.linkedin.com/in/adrianocansian/)



Curso de Redes de Computadores

Adriano Mauro Cansian
adriano.cansian@unesp.br

Capítulo 3 Camada de Transporte **(Parte 2)**

Licença de uso deste material (1)



Estes slides foram produzidos e são disponibilizados sob a licença ["Creative Commons Atribuição Não Comercial SemDerivações 4.0 Internacional \(CC BY-NC-ND 4.0\)"](#).

Como dar o crédito apropriado:

Para qualquer reutilização ou distribuição, você deve deixar claro a terceiros os termos da licença a que se encontra submetida esta obra, e seu autor. A melhor maneira de fazer isso é citando a fonte e contato. Por exemplo:

Fonte: Adriano Mauro Cansian. "Redes de Computadores I – Camada de Transporte – Parte 2", São Paulo, SP. (2025).

Contato: adriano.cansian@unesp.br. Licenciamento Creative Commons [\(CC BY-NC-ND 4.0\)](#)

Fair Use: Este material pode usar citações e figuras de outros autores. Direitos reservados aos devidos autores. Todas as citações feitas em caráter didático.

Avisos: Você não tem de cumprir com os termos da licença relativamente a elementos do material que estejam no domínio público ou cuja utilização seja permitida por uma exceção ou limitação que seja aplicável.

Não são dadas quaisquer garantias. A licença pode não lhe dar todas as autorizações necessárias para o uso pretendido. Por exemplo, outros direitos, tais como direitos de imagem, de privacidade ou direitos morais, podem limitar o uso do material.

Licença de uso deste material (2)



- **Você tem o direito de:**

Compartilhar - copiar e redistribuir o material em qualquer suporte ou formato.

- **De acordo com os termos seguintes:**

- **(BY) Atribuição** - Você deve dar o crédito apropriado, prover um *link* para esta licença e indicar se mudanças foram feitas. Você deve fazê-lo em qualquer circunstância razoável, mas de nenhuma maneira que sugira que o licenciante apoia você ou o seu uso.
- **(NC) Não Comercial** - Você não pode usar o material para fins comerciais.
- **(ND) Sem Derivações** - Se você remixar, transformar ou criar a partir do material, você não pode distribuir o material modificado.
- **Sem restrições adicionais** - Você não pode aplicar termos jurídicos ou medidas de caráter tecnológico que restrinjam legalmente outros de fazerem algo que a licença permita.

Referência para estudo:

REDES DE COMPUTADORES E A INTERNET - 6ª Edição

James F. Kurose e Keith W. Ross. Edit. Pearson 2014.

ISBN: 9788581436777.

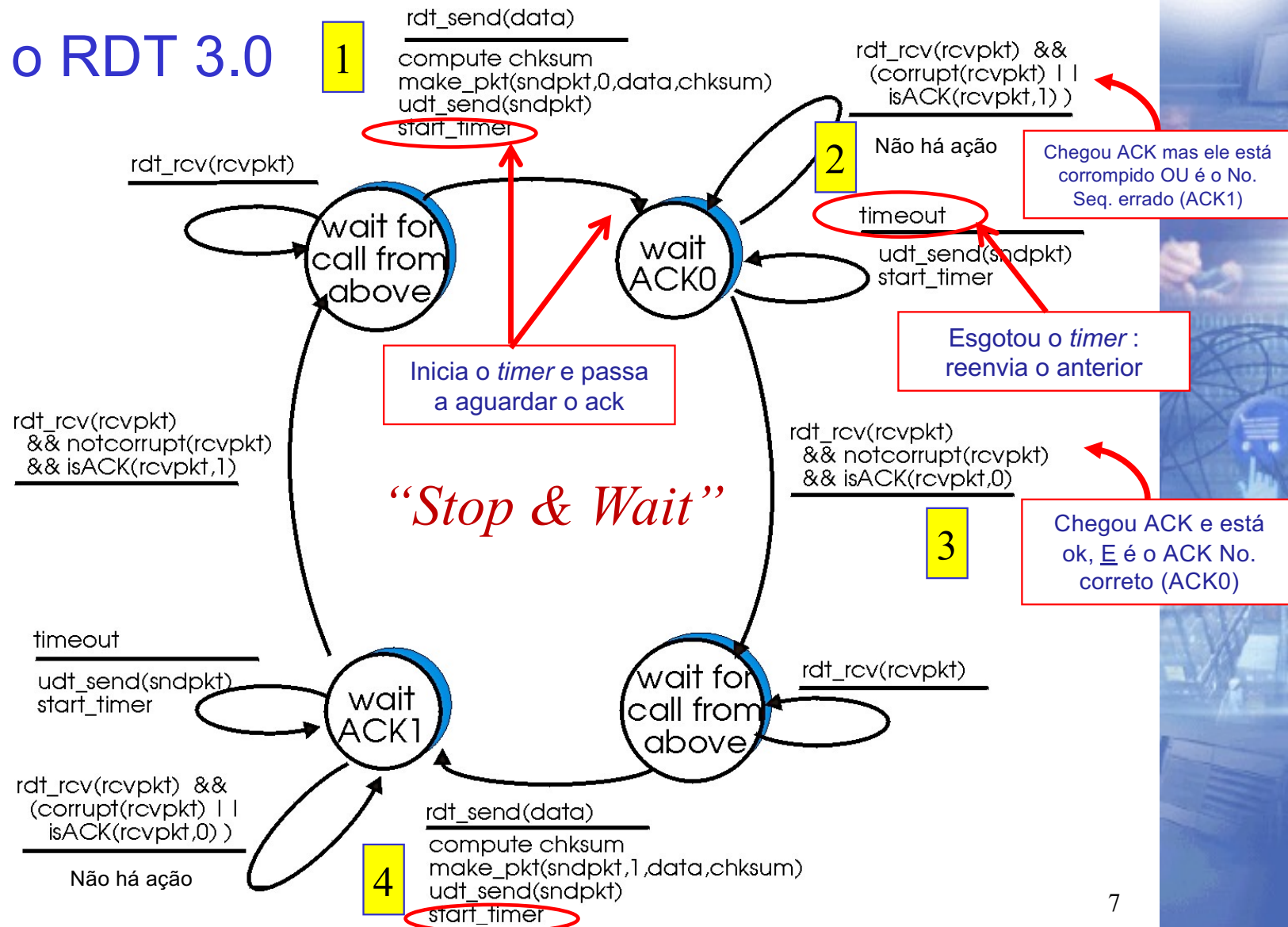
Capítulo 3 – **Camada de Transporte** (6a. Ed.)



Protocolos “dutados” (*pipelined*)

Lembrando o RDT 3.0

emissor



Problema com protocolo *stop & wait*

Em longas distâncias:

- **Atraso fim-a-fim é grande.**
- Pacote pode estar em trânsito, e **ainda não confirmado.**

No nosso modelo RDT atual: emissor fica **bloqueado** até receber o ACK do pacote anterior.



Desempenho de rdt3.0

- Desempenho do Emissor (U_{sender})

Relação entre o tempo para transmitir o pacote, e o tempo para receber a confirmação.

$$d_{trans} = \frac{L}{R}$$

Atraso de Transmissão:
Tamanho do Pacote / largura de banda

$$U_{sender} = \frac{L/R}{RTT + L/R}$$

Desempenho do EMISSOR:
Relação entre o atraso de transmissão e o tempo que ele fica bloqueado.

Tempo que o emissor fica **bloqueado**, sem poder transmitir:
= tempo de ida e volta, mais o atraso de transmissão.

Desempenho de rdt3.0

- Exemplo – considere 1 pacote de 1 KB:
 - Link de 1 Gbps (10^9 bits/seg);
 - Pacote de 1 KByte (arredondado para 8K bits)
 - Retardo fim-a-fim de **15 ms** :

Tempo de ida e volta
= 30 ms + 8 microseg

$$d_{\text{transmissão}} = \frac{8\,000 \text{ bits}}{10^9 \text{ bits/seg}} = 8 \text{ microseg}$$

$$d_{\text{trans}} = \frac{L}{R}$$

$$\text{Desempenho} = \frac{8 \text{ microseg}}{30.008 \text{ mseg}} = 0,00027 = 0,027 \%$$

$$U_{\text{sender}} = \frac{L/R}{RTT + L/R}$$

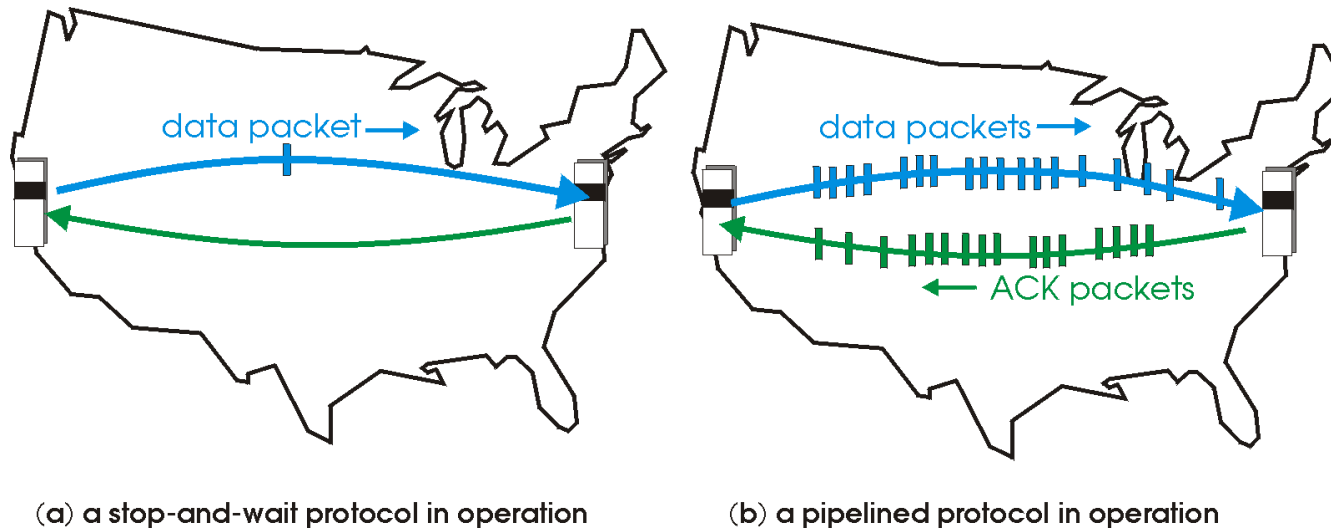
RDT 3.0 funciona, porém seu desempenho **é muito ruim**.

Um Pacote de 1KB a cada 30 mseg:
Corresponde a vazão de **264 Kbps** num enlace de 1 Gbps.
Conclusão: Protocolo limita uso dos recursos físicos.

Protocolos “dutados” (*pipelined*)

Permite existir Múltiplos pacotes **em trânsito**, ainda não confirmados.

- **Faixa de Nos. de sequência deve ser aumentada.**
- Exigirá **Buffers** no remetente e no receptor.



❑ Dois protocolos “dutados”:

- **GBN – Go Back N**
- **Retransmissão Seletiva (SR - Selective Repeat).**

GBN - Go Back N (1)

Permite o **EMISSOR** enviar múltiplos pacotes, **sem aguardar ACK do receptor.**

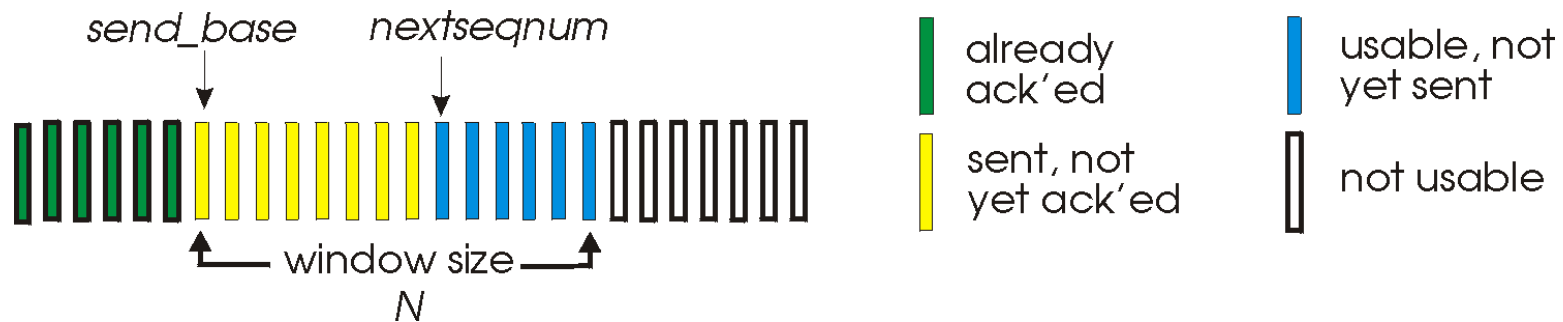
□ Entretanto:

Não poderá haver mais do que um máximo de **N** pacotes **não confirmados** no canal.

GBN - Go Back N (2)

send_base = Seq. # do pacote mais antigo não confirmado no duto.

nextseqnum = Menor Seq. # não usado (Seq.# do próximo a enviar).



$[0, \text{send_base} - 1]$ = Pacotes transmitidos e confirmados (verde)

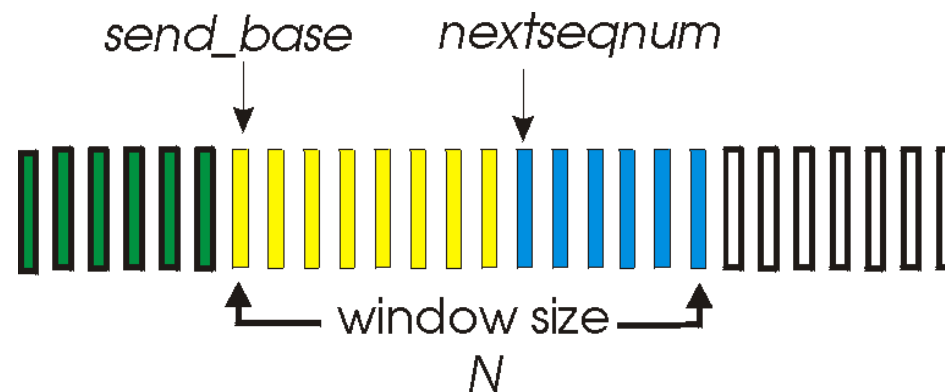
$[\text{send_base}, \text{nextseqnum} - 1]$ = Pacotes enviados mas não confirmados (amarelo)

$[\text{nextseqnum}, \text{send_base} + N - 1]$ = Disponíveis para serem usados imediatamente, assim que dados chegarem da camada superior (azul).

Sequence number $\geq (\text{send_base} + N)$: não podem ser usados até que um pacote não confirmado que esteja atualmente no duto tenha sido confirmado (brancos).

GBN - Go Back N (3)

- ❑ **No EMISSOR:** intervalo de números transmitidos, **mas ainda não confirmados**, poder ser visto como uma **janela de tamanho N** sobre o intervalo de números de sequência.
- ❑ À medida que o protocolo opera, **a janela se desloca** sobre o espaço de números de sequência: ***sliding-window protocol* = Janela deslizante**.
Pergunta: por que limitar o número de pacotes não confirmados ? (exercício)



GBN: FSM estendida do EMISSOR (1/3)

Qdo. recebe um ACK(n):
confirma que todos
pacotes até, e
inclusive, o No. de
sequência n foram
recebidos no receptor:
“**ACK cumulativo**”.
Permite receber ACKs
duplicados.

1

```
rdt_send(data)
if (nextseqnum < base+N) {
  compute chksum
  make_pkt(sndpkt(nextseqnum), nextseqnum, data, chksum)
  udt_send(sndpkt(nextseqnum))
  if (base == nextseqnum)
    start_timer
  nextseqnum = nextseqnum + 1
}
else
  refuse_data(data)
```

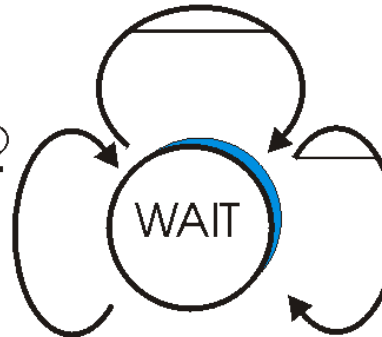
2

Chamada superior:
verifica se a janela está cheia. Se não está,
forma o pacote e envia.
Se está cheia, recusa.

Temporizador *Timeout(n)*:
retransmite pacote n e **todos** os
pacotes com No. de sequência
maiores na janela.

3

```
rdt_rcv(rcv_pkt) && notcorrupt(rcvpkt)
base = getacknum(rcvpkt)+1
if (base == nextseqnum)
  stop_timer
else
  start_timer
```



```
timeout
start_timer
{
  udt_send(sndpkt(base))
  udt_send(sndpkt(base+1))
  .....
  udt_send(sndpkt(nextseqnum-1))
}
```

Veja o próximo slide

GBN: FSM estendida do EMISSOR (2/3)

2

Se ocorrer um **esgotamento do temporizador**, o emissor **reenvia TODOS os pacotes** que **tinham sido previamente enviados**, mas que **ainda não tinham sido reconhecidos**.

rdt_send(data)

```
if (seqnum < base+N) {  
    calculate checksum  
    udt_send(sndpkt(nextseqnum), nextseqnum, data, checksum)  
    seqnum = nextseqnum  
    timer = nextseqnum + 1  
}
```

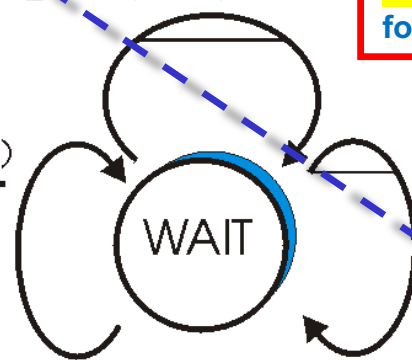
1

Temporizador: é um cronômetro para o **pacote mais antigo transmitido**, mas que **ainda não foi confirmado**.

refuse_data(data)

rdt_rcv(rcv_pkt) && notcorrupt(rcvpkt)

```
base = getacknum(rcvpkt)+1  
if (base == nextseqnum)  
    stop_timer  
else  
    start_timer
```

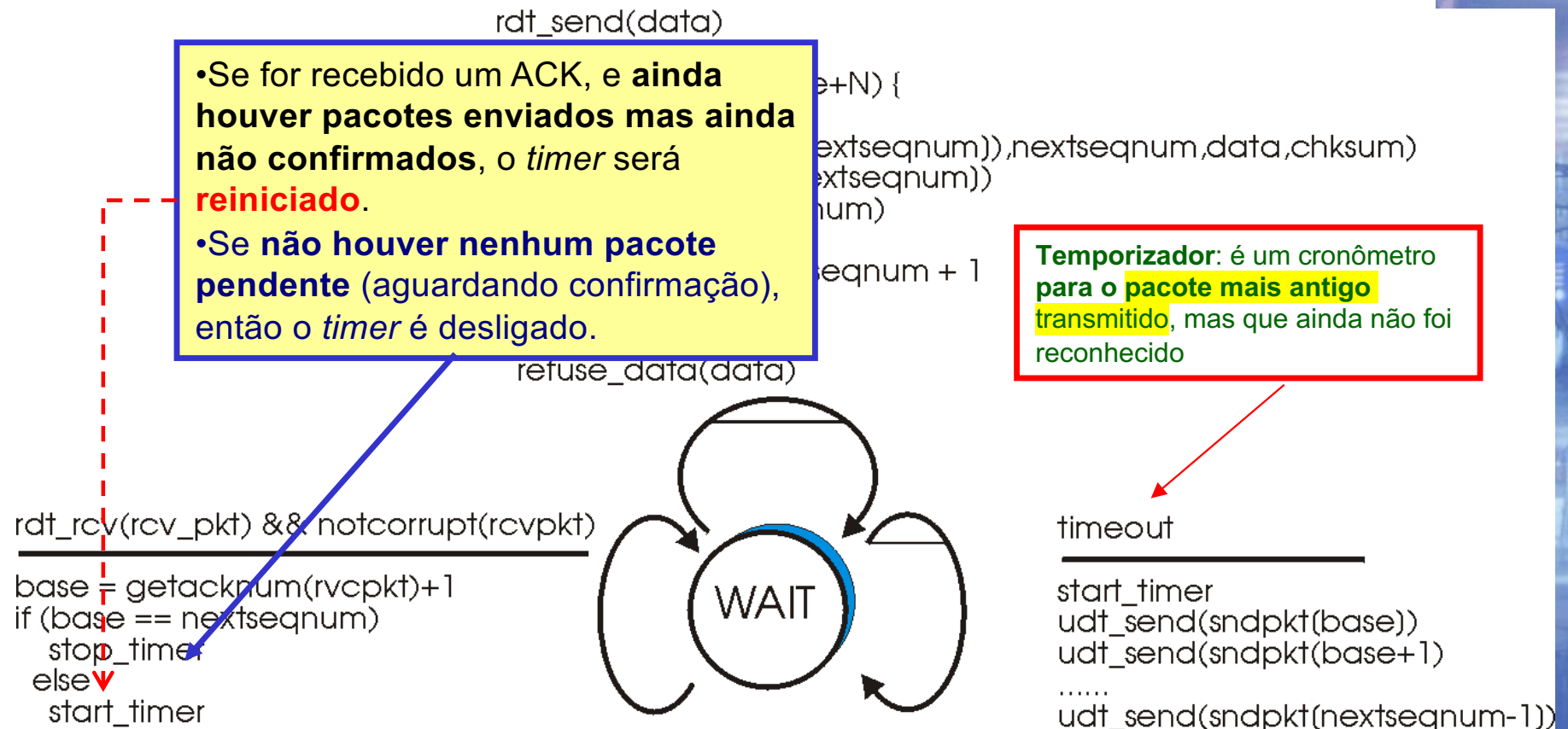


timeout

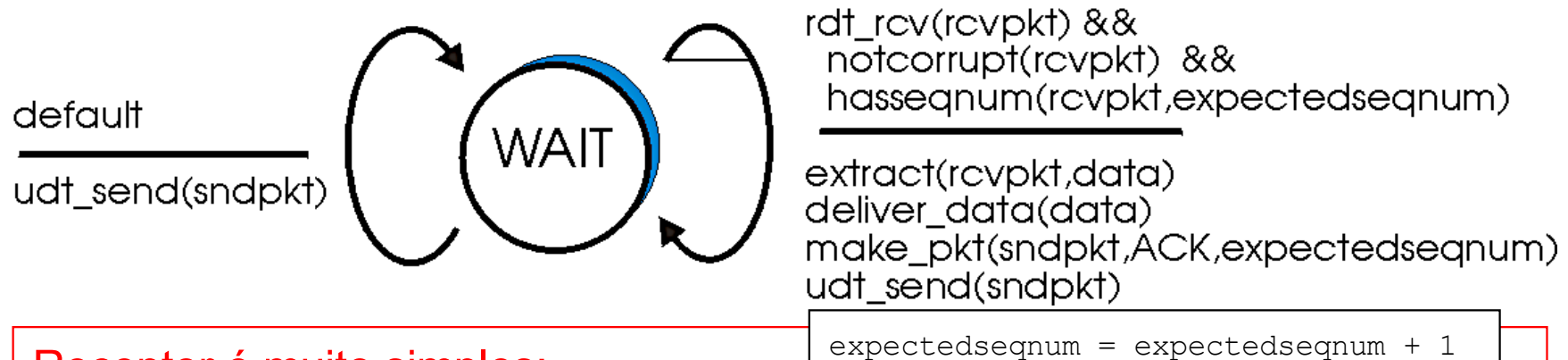
```
start_timer  
udt_send(sndpkt(base))  
udt_send(sndpkt(base+1))  
.....  
udt_send(sndpkt(nextseqnum-1))
```

Veja o próximo slide

GBN: FSM estendida do EMISSOR (3/3)



GBN: FSM estendida do **RECEPTOR**



Receptor é muito simples:

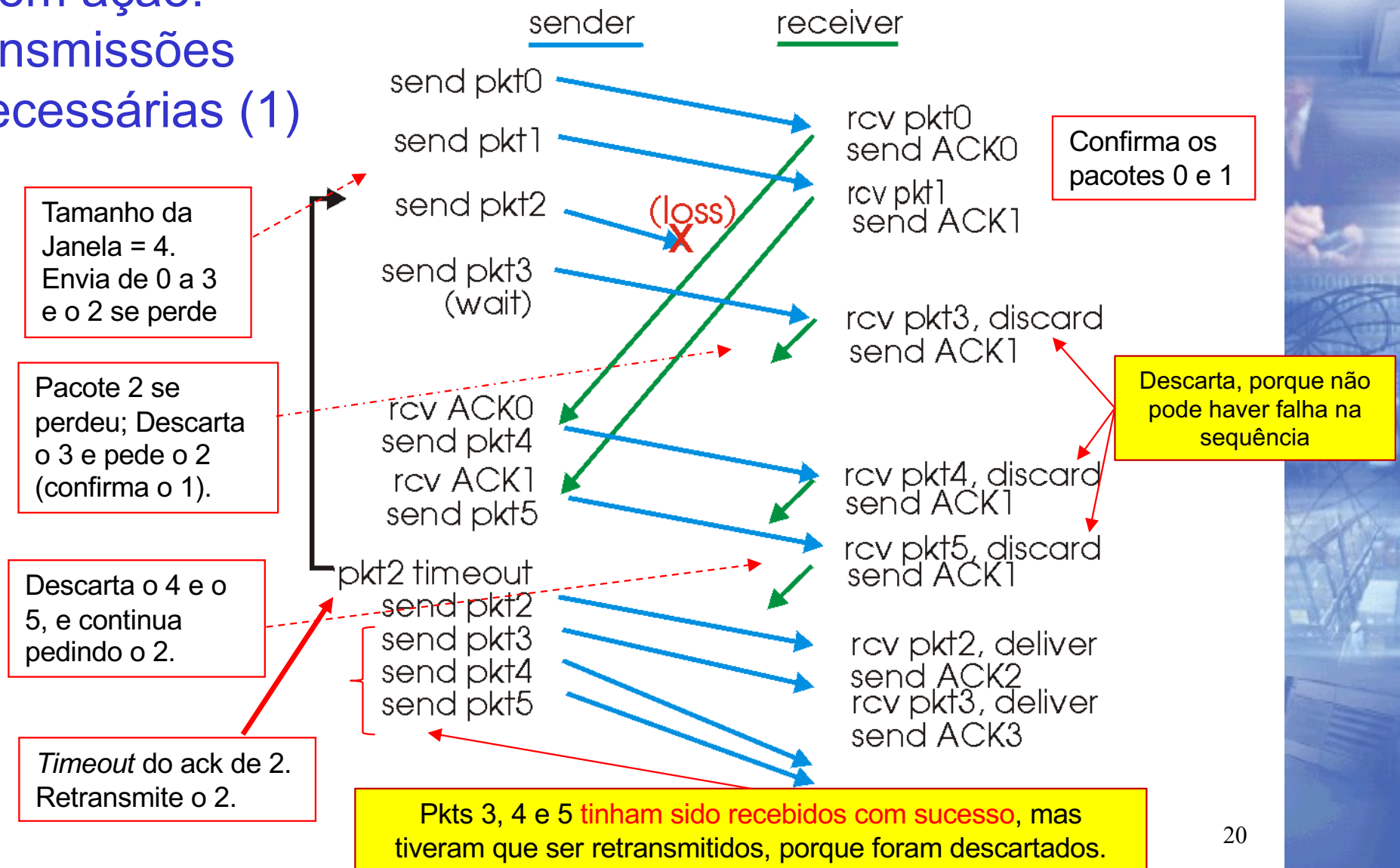
- ❑ Usa apenas ACK: sempre envia ACK para pacote recebido o.k. com o maior número de sequência **em ordem**.
 - Pode gerar ACKs duplicados
 - Só precisa se lembrar do **expectedseqnum** (Número de Seq. esperado).
- ❑ Se recebe Pacote **fora de ordem**:
 - Descarta (não armazena) → receptor não usa *buffers* (TCP usa!).
 - Manda ACK de pacote com maior No. de sequência em ordem.

Problemas do GBN: performance

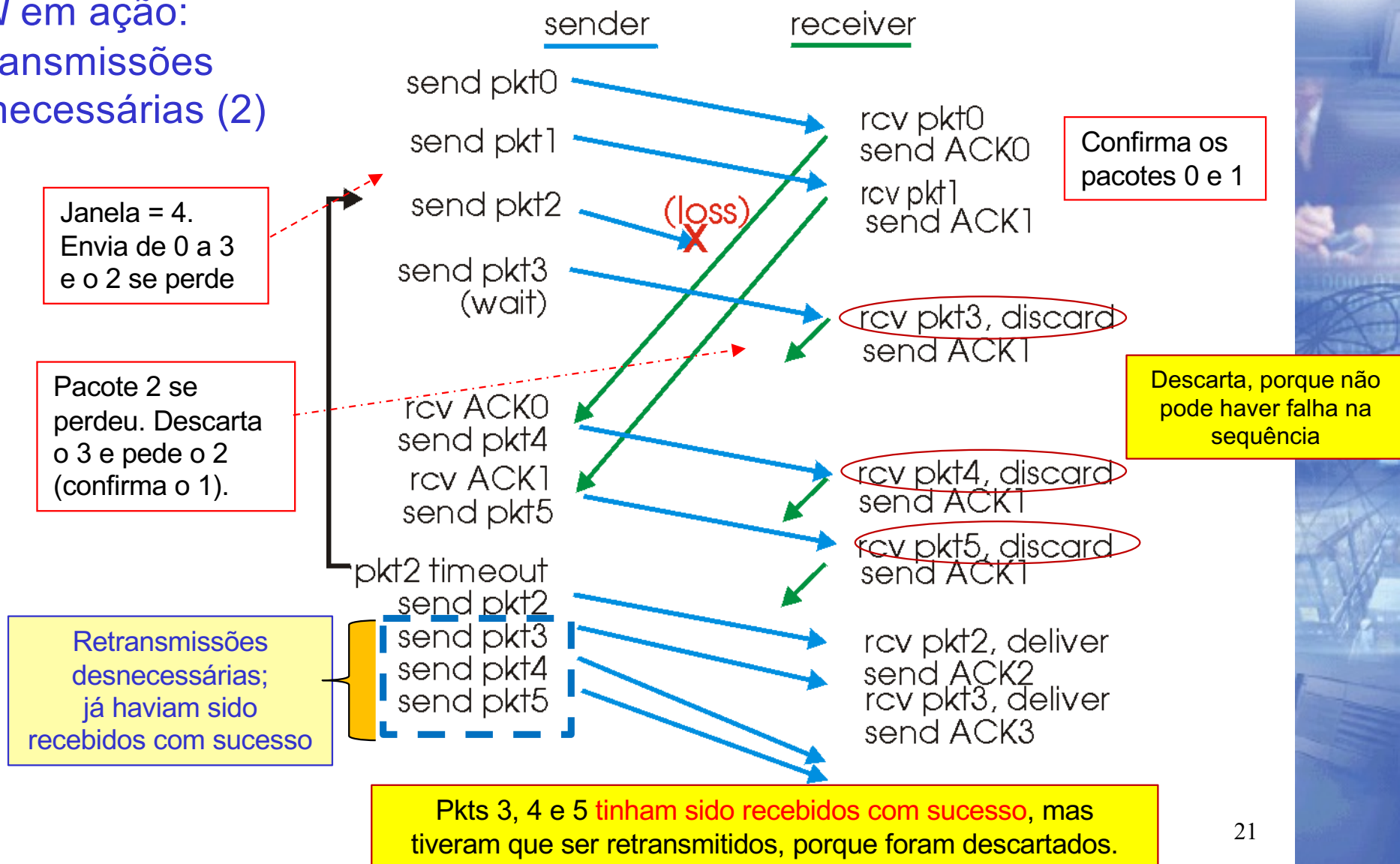
Se o tamanho da janela é grande
E
atraso da rede também é grande:
muitos pacotes podem estar no duto.

- ❑ Erro em um único segmento:
 - Resulta na **retransmissão de um grande número de segmentos**, a maioria desnecessários.
- ❑ À medida em que a probabilidade de **erro do canal cresce**, **o duto fica lotado de retransmissões desnecessárias.**

GBN em ação: Retransmissões desnecessárias (1)



GBN em ação: Retransmissões desnecessárias (2)



Mas...*GBN* não é o TCP

- ❑ GBN incorpora **quase** todas as técnicas que serão encontradas nos componentes do TCP (visto mais adiante):
 - Número de sequência.
 - *Checksum*.
 - *ACK* cumulativos.
 - *Timeouts*.
 - Operação de retransmissão.
- ❑ Entretanto existem diferenças entre o TCP e o *GBN*.
- ❑ Por exemplo: implementações TCP fazem *buffering* de segmentos recebidos corretamente, mas fora de ordem.
- ❑ TCP é um híbrido de **GBN** e **Repetição Seletiva** (a seguir).

Repetição Seletiva

Selective Repeat

Repetição seletiva (1/2)

- ❑ Evita retransmissões desnecessárias.

Emissor: retransmite apenas os pacotes que suspeita terem sido **recebidos com erro** pelo receptor.

- ❑ **Receptor:** confirma **individualmente** todos os pacotes **recebidos** corretamente.

Armazena pacotes no **buffer**, se necessário, para posterior **entrega em ordem** à camada de aplicação.

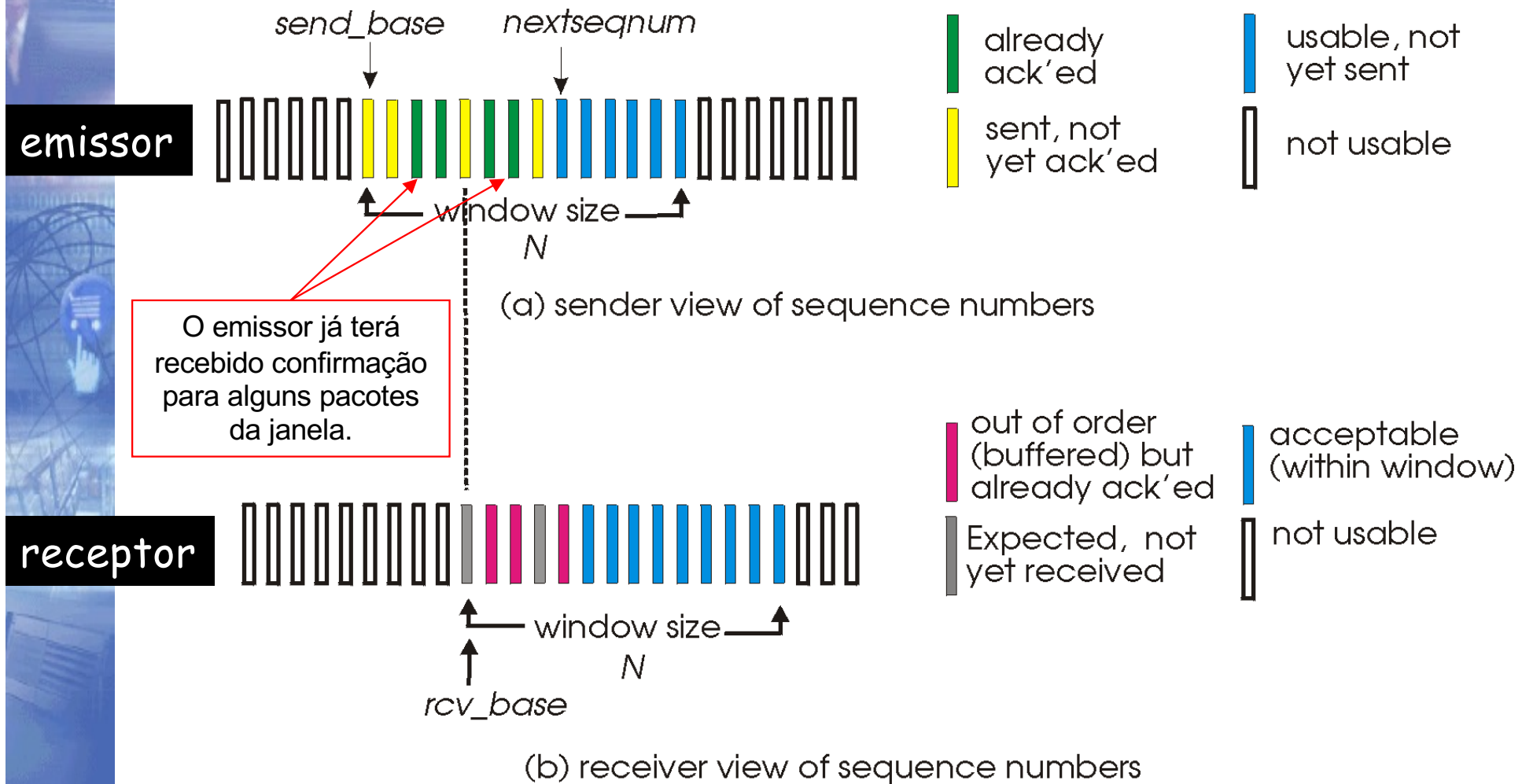
Repetição seletiva (2/2)

- Uma vez que o **emissor** apenas re-envia pacotes para os quais o ACK não foi recebido → Complica um pouco:

É necessário um Timer de EMISSOR para cada pacote sem ACK.

- Janela do emissor:
 - Pode usar **N** números de sequência consecutivos.
 - Deve **limitar os Nos. de sequência** de pacotes enviados, mas ainda não confirmados.

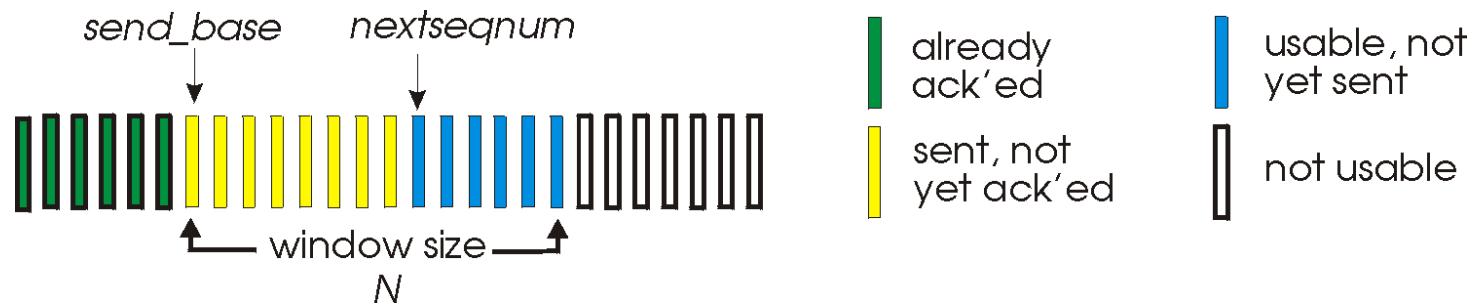
Repetição seletiva: janelas de emissor e receptor



Comparando com *GBN - Go Back N*:

send_base = Seq. # do pacote mais antigo não confirmado no duto.

nextseqnum = Menor Seq. # não usado (Seq.# do próximo a enviar).



Repetição seletiva no **EMISSOR** (1/2)

□ **Dados recebidos de acima:**

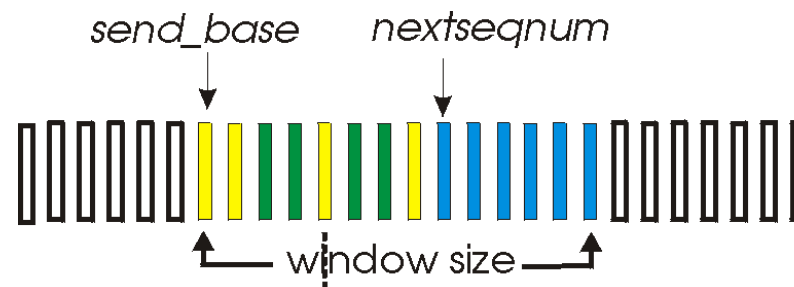
- **Emissor** verifica o próximo No. sequência disponível.
- Se o No. de sequência **estiver dentro da janela** do emissor, os dados são empacotados e enviados.
 - Caso contrário são *bufferizados*, ou devolvidos à camada superior para transmissão posterior.

□ **Timeout:** temporizadores são usados para proteger contra perda de pacotes.

- **Somente um único pacote será transmitido no caso de *timeout*.**

Repetição seletiva no **EMISSOR** (2/2)

- ❑ **ACK recebido**: **emissor** marca o pacote como recebido.
 - Se o número de sequência do pacote for igual à **send_base**, então a **base da janela avança até o pacote com o menor número de sequência não-confirmado**.
 - Se houver pacotes não transmitidos, **com números de sequência que caem agora dentro da janela**, estes pacotes são transmitidos.

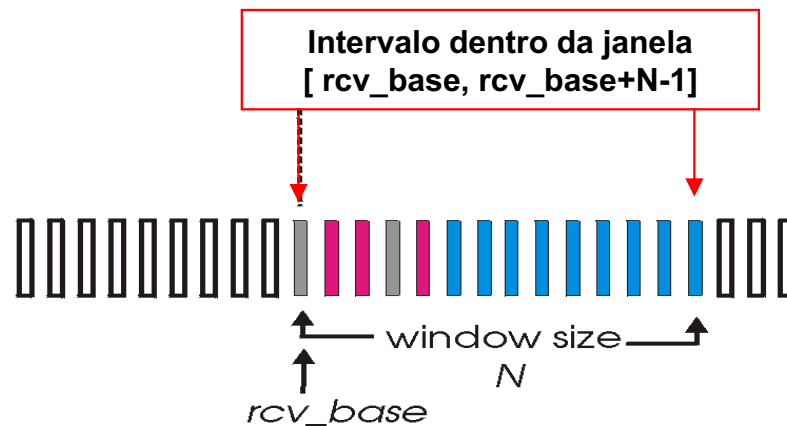


Eventos de SR no **receptor** (1/2)

1.) Pacote com número de sequência no intervalo

[rcv_base, rcv_base+N-1] é recebido corretamente:

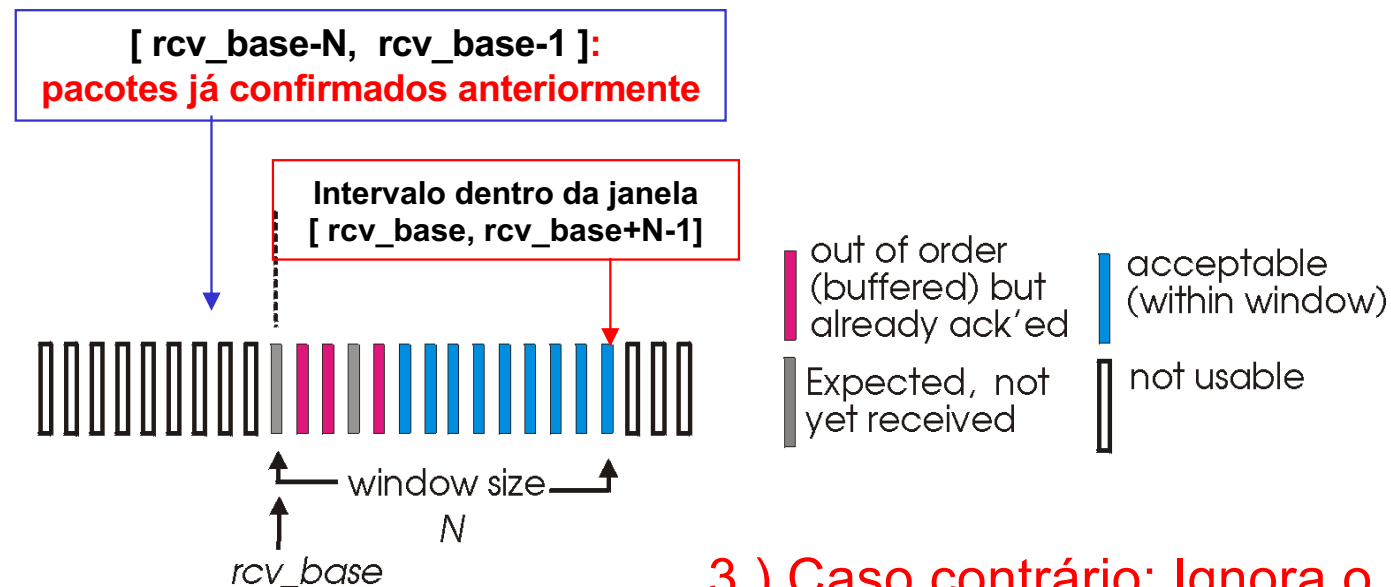
- Neste caso, o pacote recebido cai dentro da janela do receptor.
- Um **ACK seletivo** é retornado ao remetente.
- Se o pacote não foi recebido previamente, ele é **armazenado**.
- Se este pacote tiver um número de sequência **igual à base da janela** da recepção (**rcv_base**), então este pacote, e os quaisquer pacotes previamente armazenados e numerados em sequência (começando com o **rcv_base**) **são entregues à camada superior**.
- A **janela de recepção é movida** então para a frente pela quantidade do número total dos pacotes entregues à camada superior.



(b) receiver view of sequi

Eventos de SR no **receptor** (2/2)

2.) Pacote com número de sequência é recebido dentro de $[rcv_base - N, rcv_base - 1]$: Neste caso, um ACK deve ser gerado, mesmo que este seja um pacote que o receptor já tenha confirmado previamente.



3.) Caso contrário: Ignora o pacote.

(b) receiver view of sequence numbers

Repetição seletiva - resumo

emissor

Dados de cima:

- ❑ Se próximo No. de sequência na janela → envia o pacote.

Timeout(n):

- ❑ Reenvia pacote n .
- ❑ Reinicia o temporizador.

ACK(n) em $[sendbase, sendbase+N]$:

- ❑ Marca pacote n como “recebido”.
- ❑ Se N for **menor pacote não confirmado**, avança base da janela ao próximo No. de sequência não confirmado.

receptor

• Pacote n dentro de

$[rcvbase, rcvbase+N-1]$:

Envia ACK(n).

Fora de ordem: *buffering*.

Em ordem: entrega (tb. entrega pacotes em ordem do buffer), avança janela p/ próximo pacote ainda não recebido.

• Pacote n em

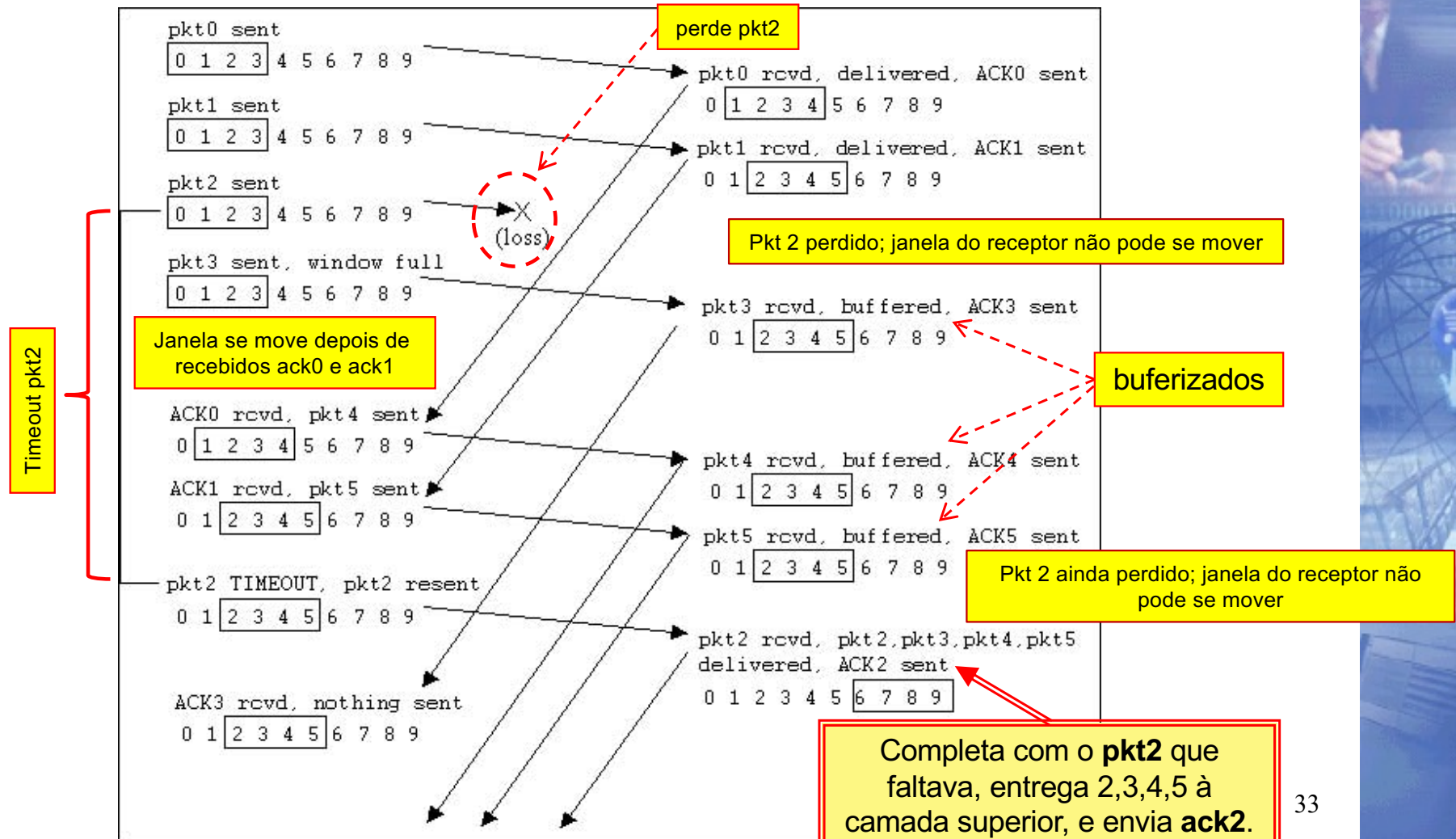
$[rcvbase-N, rcvbase-1]$

ACK(n), mesmo que já tenha enviado antes.

• Senão:

Ignora.

Retransmissão seletiva em ação



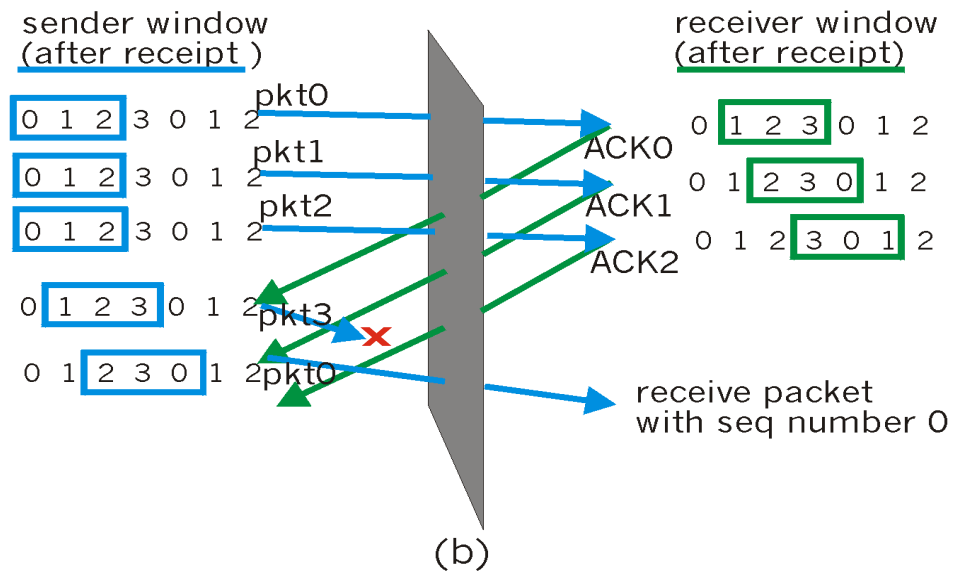
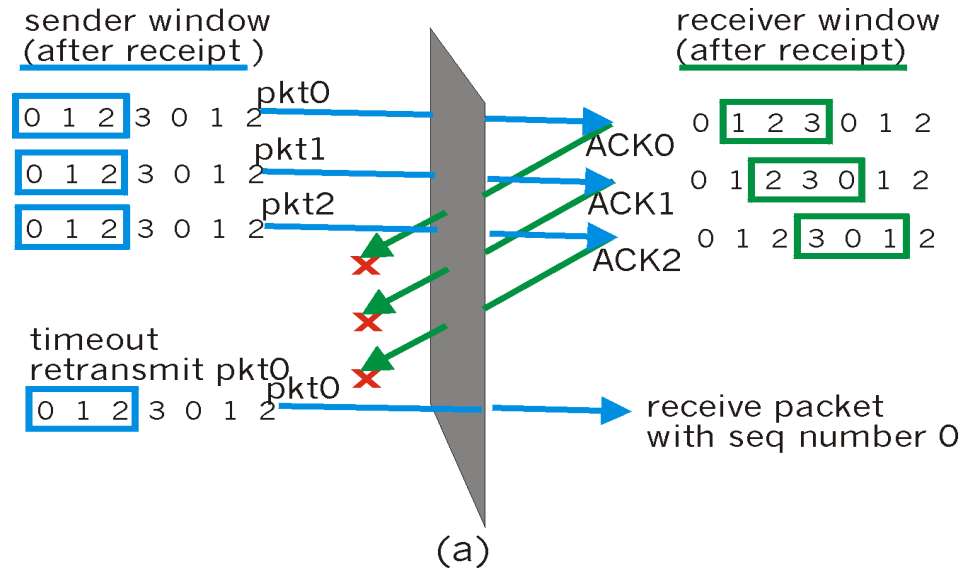
Dilema da Repetição Seletiva:

Exemplo:

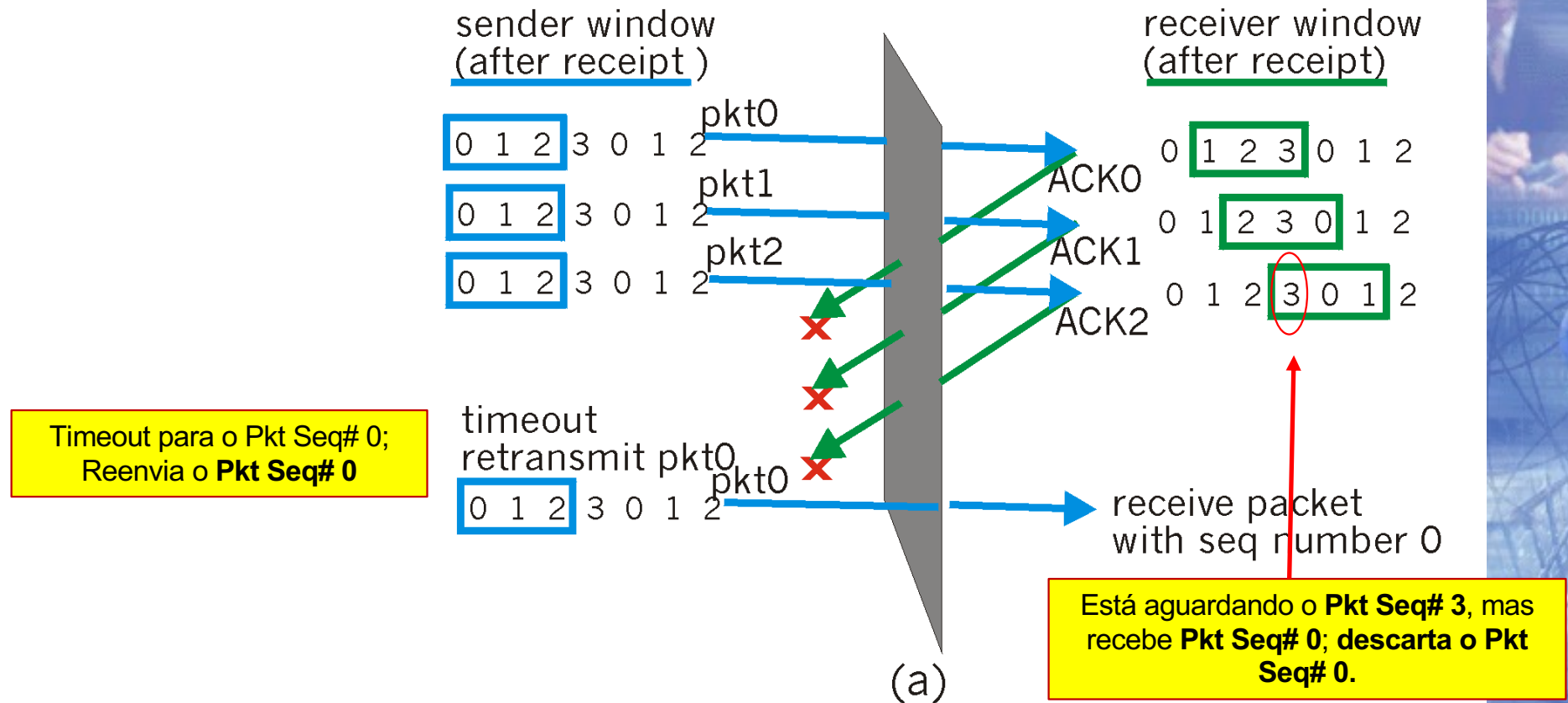
- ❑ 4 Nos. de sequência: 0, 1, 2 e 3.
- ❑ Tamanho de janela = 3.

❑ **Exercício:**

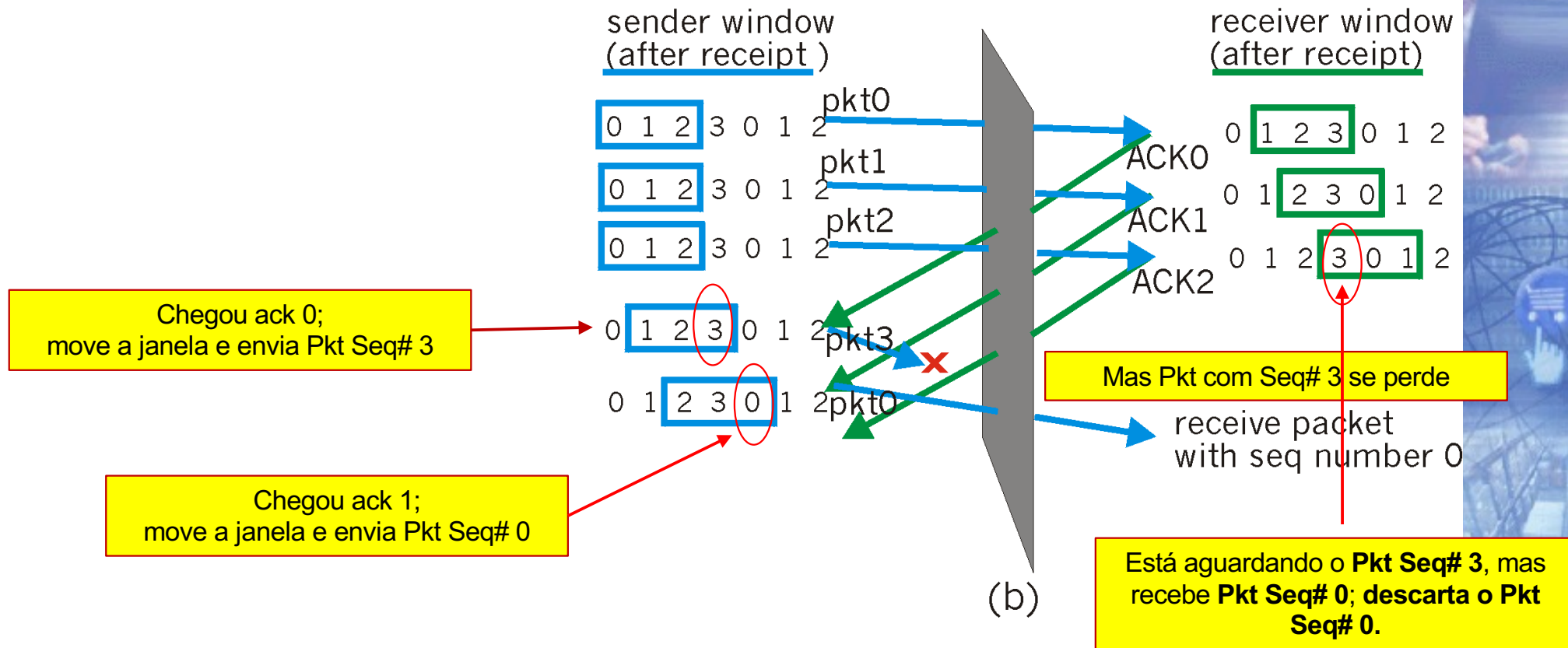
Qual a diferença entre os dois cenários (a.) e (b.) para o **receptor**?



Repetição seletiva – dilema – cenário a.)



Repetição seletiva – dilema – cenário b.)

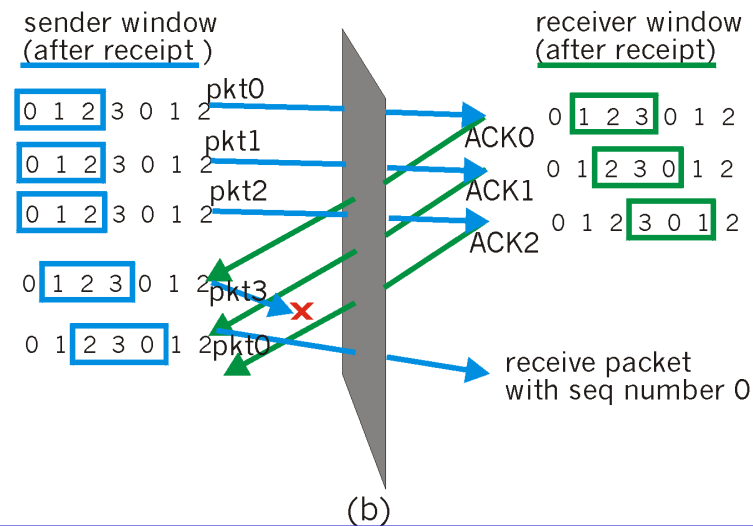
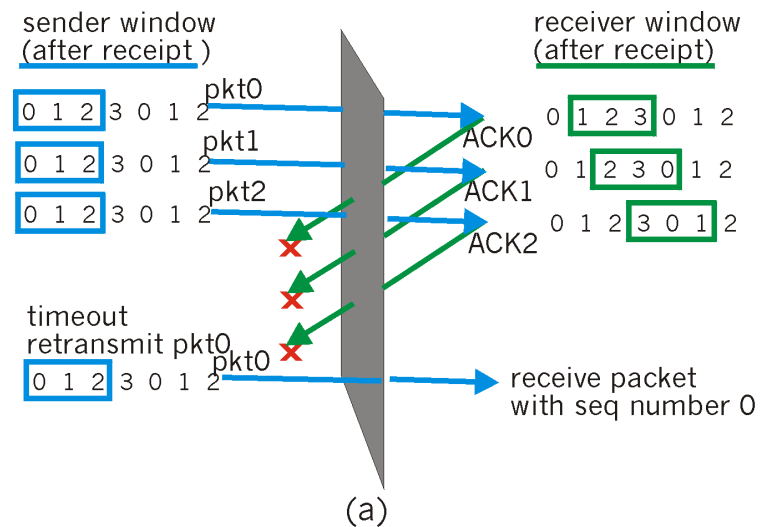


Repetição seletiva: dilema

Exemplo:

- ❑ 4 Nos. de sequência : 0, 1, 2 e 3.
- ❑ Tamanho de janela = 3.
- ❑ **Receptor não vê diferença entre os dois cenários (a.) e (b.) !!!**

Pergunta / exercício: Qual a relação entre tamanho de No. de sequência e tamanho de janela?



Conclusão: Um tamanho de janela que seja igual ao tamanho de numeração sequencial, menos 1, não vai funcionar.

O TCP

A implementação do TCP

TCP: Visão geral (1) RFCs: 793, 1122, 1323, 2018, 2581

❑ Processo-a-processo:

- *Peer process* fala diretamente com outro *peer process*.

❑ Entrega fluxo de bytes **ordenados** e confiável:

❑ Dutado (*pipelined*):

- Tamanho da janela ajustado por controle de fluxo e congestionamento do TCP.

❑ Usa *Buffers* de envio e recepção, e variáveis de estado para cada conexão.

TCP: Visão geral (2)

RFCs: 793, 1122, 1323, 2018, 2581

❑ O tráfego é *Full Duplex*:

- Fluxo de dados **bi-direcional** na **mesma conexão**.
- MSS: é o tamanho máximo de segmento **de dados**.

(Falaremos de MSS mais adiante)

1500 bytes, 536
bytes ou 512 bytes

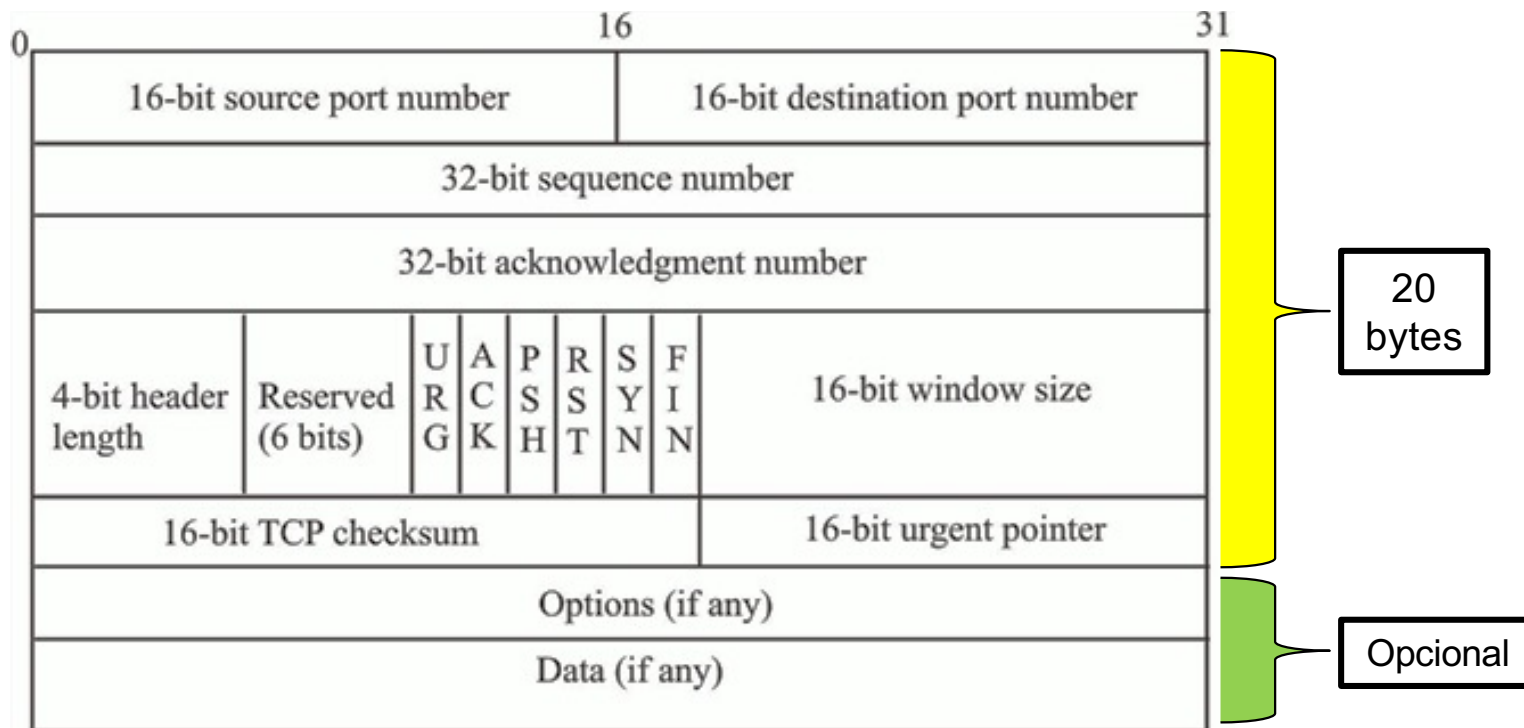
❑ É orientado a conexão:

- *Handshaking* de **3 vias** (troca de msgs de controle) inicia estado de remetente, receptor antes de trocar dados.

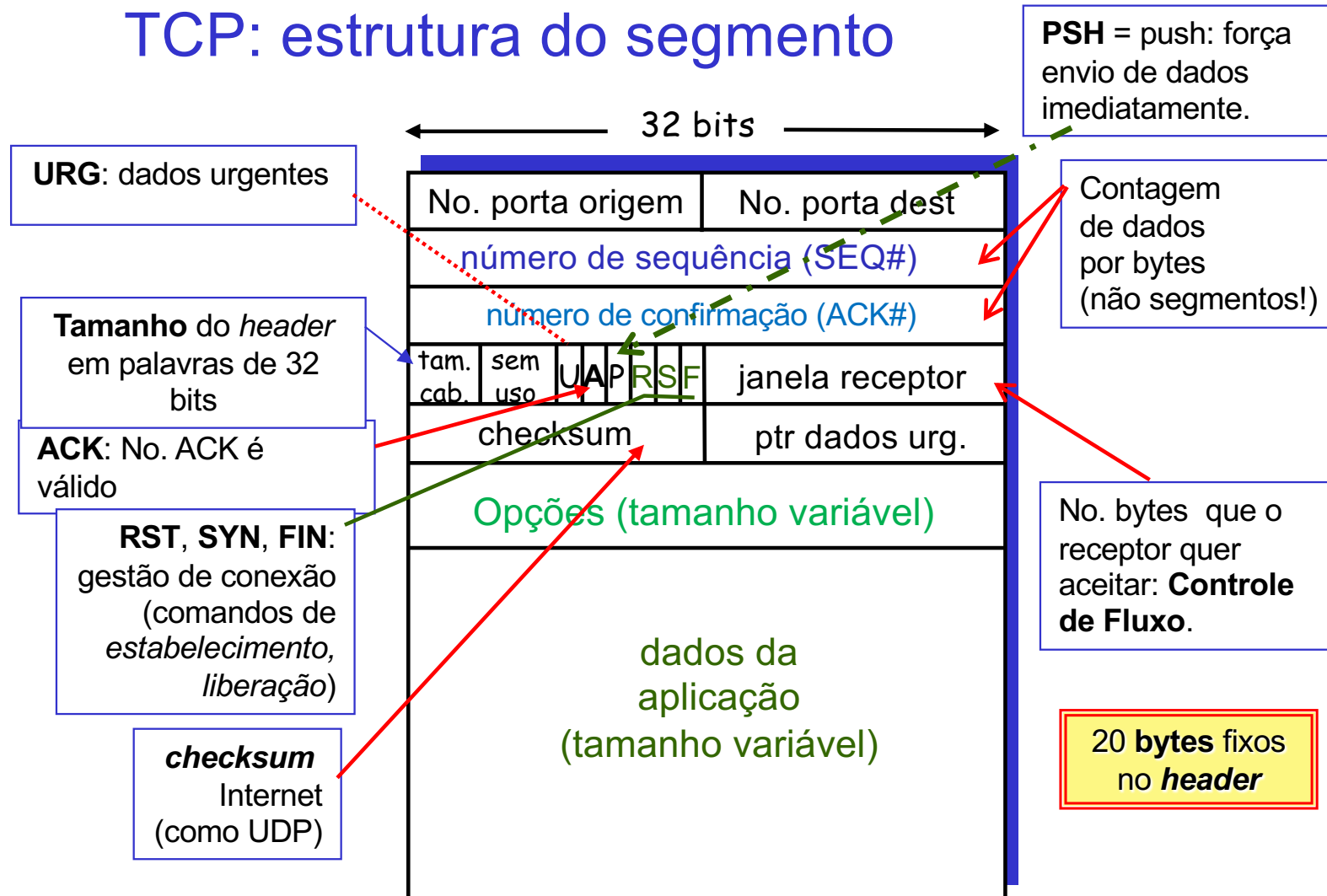
❑ Tem o **Fluxo controlado**: o receptor não será afogado.

(veremos controle de fluxo mais adiante)

TCP Header



TCP: estrutura do segmento



Header TCP

- ❑ **Porta Origem e Porta Destino** identificam o processo de aplicação que está enviando dados e o processo de aplicação que irá receber os dados.
- ❑ **Número de sequência (SEQ#)** identifica os bytes enviados. Na prática ele é a identificação do primeiro byte de dados contido no segmento enviado. Os demais são seqüenciados a partir deste byte.
- ❑ **Acknowledgement (ACK#)** identifica os bytes que foram recebidos e tratados sem erro pelo destino, bem como a sequência do próximo byte esperado.
- ❑ **Tamanho header** : número decimal de palavras de 32 bits do header TCP tipicamente contém valor 5 (4 bits).
- ❑ **Reservado** é um campo ainda não utilizado **(6 bits) → veja exercício no final deste material**
- ❑ **FLAGS** identifica as *flags* especiais de operação (U-urg | A-ack | P-push | , R-rst | S-syn | F-fin)
- ❑ **Window** identifica o tamanho da janela para o controle de fluxo
- ❑ **Checksum** destina-se a verificação de erros de transmissão. É calculado usando o pseudo header, o header TCP e também a área de dados.
- ❑ **Urgent Pointer** é um ponteiro para dados urgentes, contidos na área de dados (caso bit **U** setado para 1).

MSS e numeração de segmentos

MSS (*Maximum Segment Size*)

- MSS representa o tamanho do **maior bloco de dados** que o TCP permite enviar **num único segmento**.
 - MSS é **ajustado automaticamente pelo SO**.
 - *Default* (obrigatório): **536** bytes.
 - (20 bytes IP, 20 bytes TCP, para um total de 576 bytes).
 - Ethernet padrão: **1460** bytes.
 - (20 bytes IP, 20 bytes TCP, para um total de **1500 bytes**)

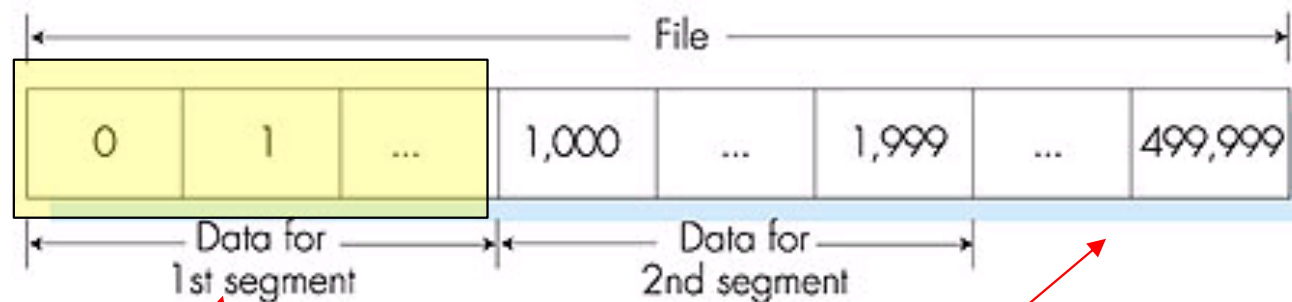
Quanto maior o MSS, melhor

□ Em geral: **quanto maior o MSS, melhor o desempenho da rede.**

- Mais dados são enviados num único pacote.
- Quanto maior a quantidade de dados enviados em um único bloco, menor o *overhead* de *headers* TCP e IP.
- Desde que não ocorra fragmentação.
(Trataremos fragmentação no protocolo IP; associado ao MTU)
- O MSS está limitado pelo MTU, que está limitado pela tecnologia ou protocolo **da camada de enlace**.
 - Esta relação será discutida mais adiante, **junto com a discussão sobre MTU na camada de rede.**

MSS e numeração de segmentos

Aplicação quer enviar 500.000 bytes de dados,
Numa camada de transporte TCP com MSS = 1.000 bytes
Transmissão TCP: 500 partes de 1.000 bytes



1o. segmento

último segmento

O No. de sequência do emissor é incrementado pelo No. de bytes enviados

TCP: Gerenciamento de Conexões

TCP – Características:

- ❑ Emissor e receptor estabelecem “conexão”.
- ❑ **Full duplex**: fluxo de dados segue nos dois sentidos.
- ❑ Inicializam variáveis TCP (*Statefull*):
 - Números de sequência e de confirmação.
 - *Buffers*, informações de controle de fluxo (por ex. **RcvWindow**) e temporizadores.
- ❑ **Cliente**: é aquele que inicia a conexão.
 - *Active Open*
- ❑ **Servidor**: é aquele chamado pelo cliente.
 - *Passive Open*.

TCP - Iniciando Conexão (1)

Inicialização em 3 passos (*3-way handshake*):

(1): **Cliente** envia segmento de **controle de início de conexão** TCP ao servidor.

- Bit **SYN** do TCP é ajustado como 1 e Bit ACK como 0. (**SYN=1; ACK=0**)
- Cliente envia seu No. de Sequência inicial Seq#C1 (32 bits)

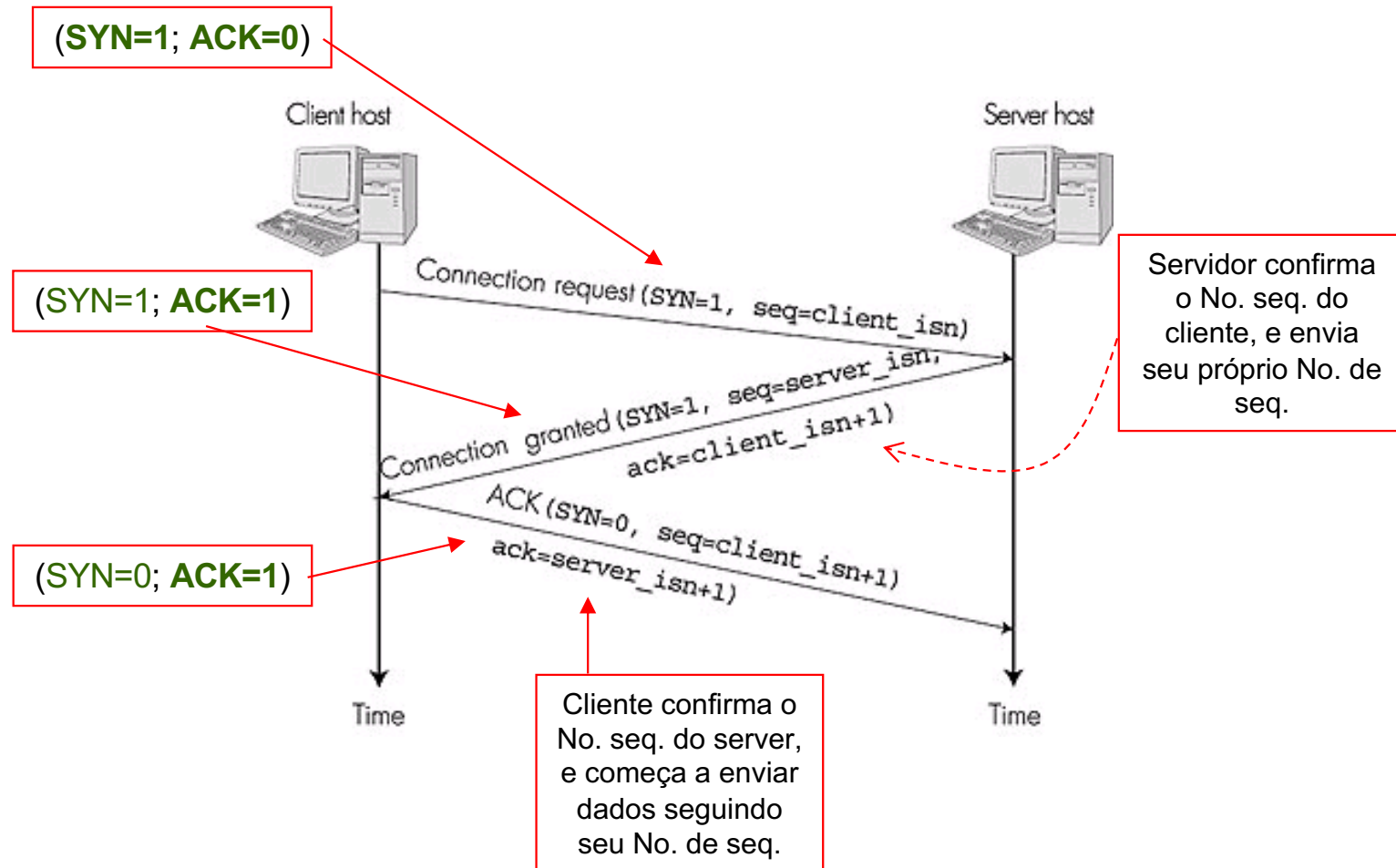
(2): **Servidor** recebe (**SYN=1, ACK=0**) e Seq#C1

- Responde com pacote setando bits **SYN=1** e **ACK=1**
- Confirma SYN# do cliente recebido setando ACK# = (Seq#C+1).
- Envia seu Seq#S1 (No. Seq. inicial do servidor)
- Aloca *buffers*, especifica No. seq. inicial de servidor para o receptor.

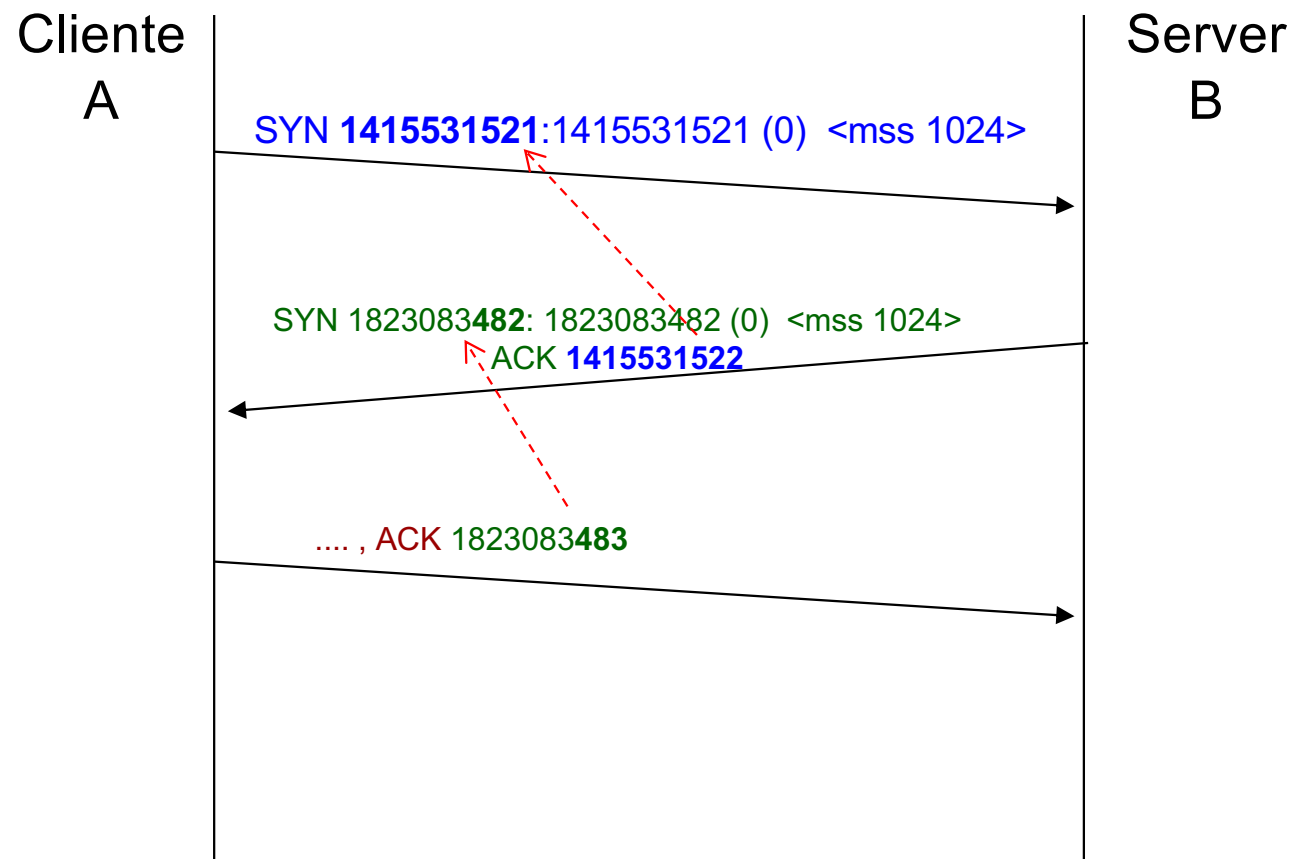
(3): **Cliente** recebe **SYN=1; ACK=1**

- Responde com segmento **ACK**, **confirmando o número de sequencia recebido**,
- **E começa a enviar dados.** (**SYN=0; ACK=1**) e SEQ# / ACK# do cliente.

TCP: Iniciando Conexão (2)

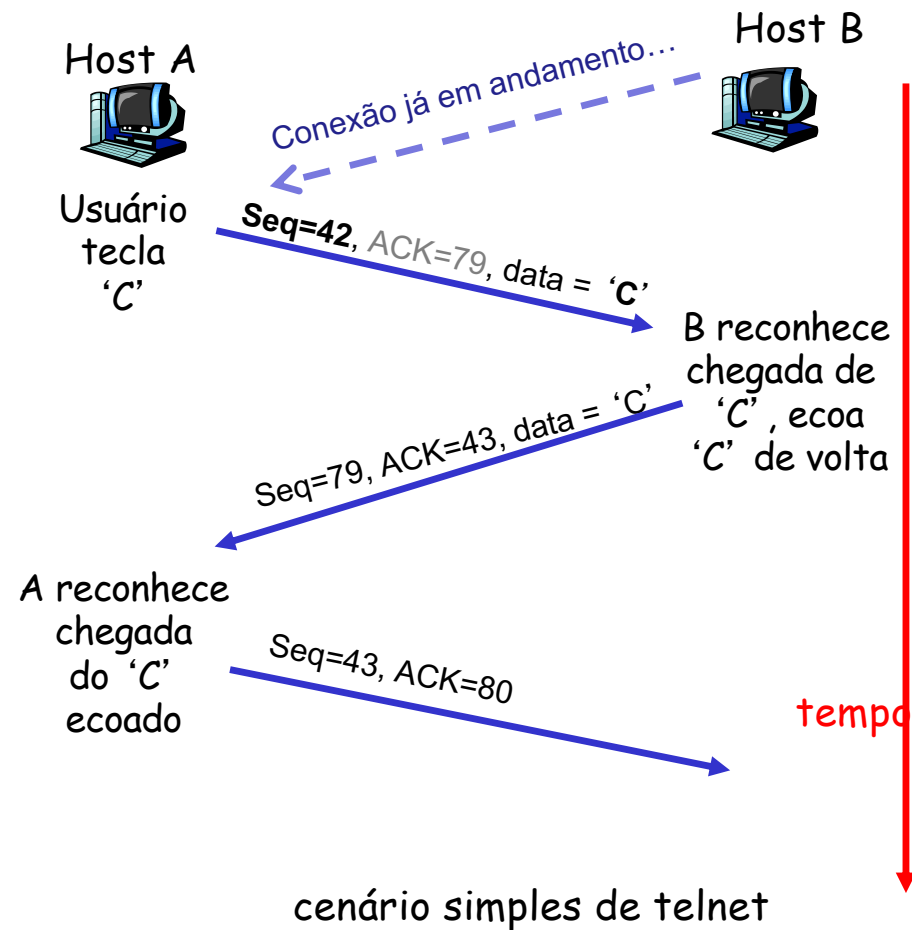


Estabelecimento da Conexão



TCP: Números de sequência e ACKs

- **Nos. de sequência:** é o “**número**” do primeiro byte de dados do segmento.
- **ACKs:** No. de sequência do **próximo segmento esperado** (**em bytes**).
 - ACK cumulativo.
- Como receptor trata segmentos fora da ordem?
 - Especificação do TCP é omissa - deixado ao implementador.
 - Maioria: buferiza.



ACKs duplicados

- ❑ *First time ack* é sinal que está tudo ok.
- ❑ Mas, o que acontece se o emissor recebe um **ACK duplicado**?
 - “**re-reconhece**” um segmento para qual o emissor já tenha recebido ACK anteriormente.
- ❑ Veremos em seguida como isso é usado pelo TCP...

Lidando com ACKs duplicados e retransmissões.

- ❑ No TCP os segmentos são enviados em paralelo (*pipeline*).
- ❑ Os ACKs são **cumulativos**.
 - **A chegada de um ACK confirma anteriores.**
- ❑ TCP usa único *timer* de retransmissão.
 - **É um *timer* para o segmento mais antigo sem ACK.**
- ❑ Retransmissões são disparadas por:
 - Eventos de *timeout*.
 - ACKs duplicados.
- ❑ Inicialmente, para facilitar o entendimento, vamos considerar um emissor TCP simplificado:
 - **Inicialmente: ignorar controle de fluxo e controle de congestionamento.**
 - Serão vistos mais adiante.

Eventos de **emissor** TCP (1)

□ Quando dados são recebidos da aplicação:

- Cria segmento com *seq#*
- O número *seq#* é a quantidade da cadeia de bytes desde o **primeiro byte de dados do segmento.**
- Inicia ***timer***, se ainda não tiver iniciado.
 - É um ***timer*** para o segmento mais antigo sem ACK.
 - Intervalo de expiração: *Time-Out-Interval*

Eventos de **emissor** TCP (2)

EVENTOS:

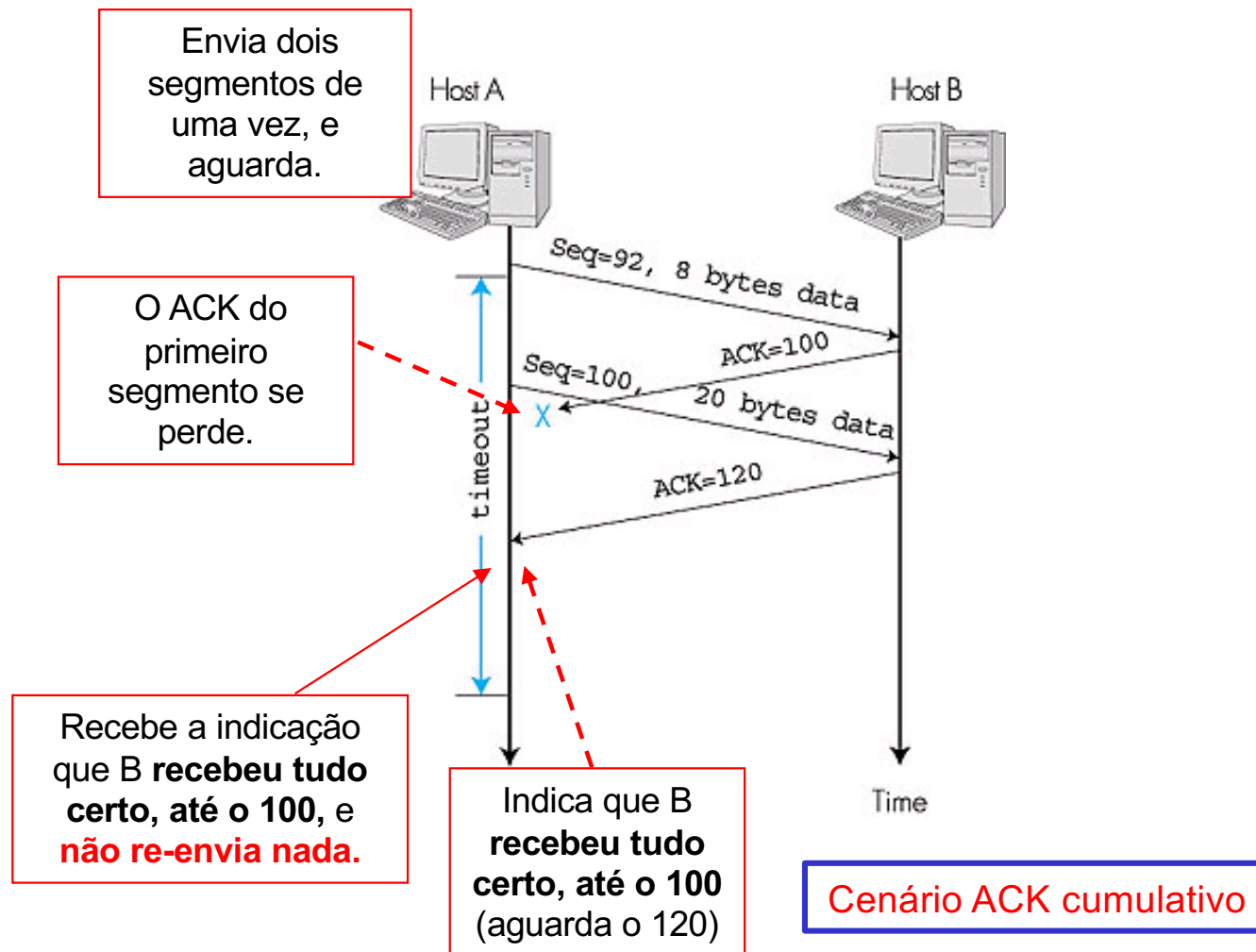
1.) Evento *Timeout*:

- ❑ Retransmite segmento que causou *timeout*.
- ❑ Reinicia *timer*.

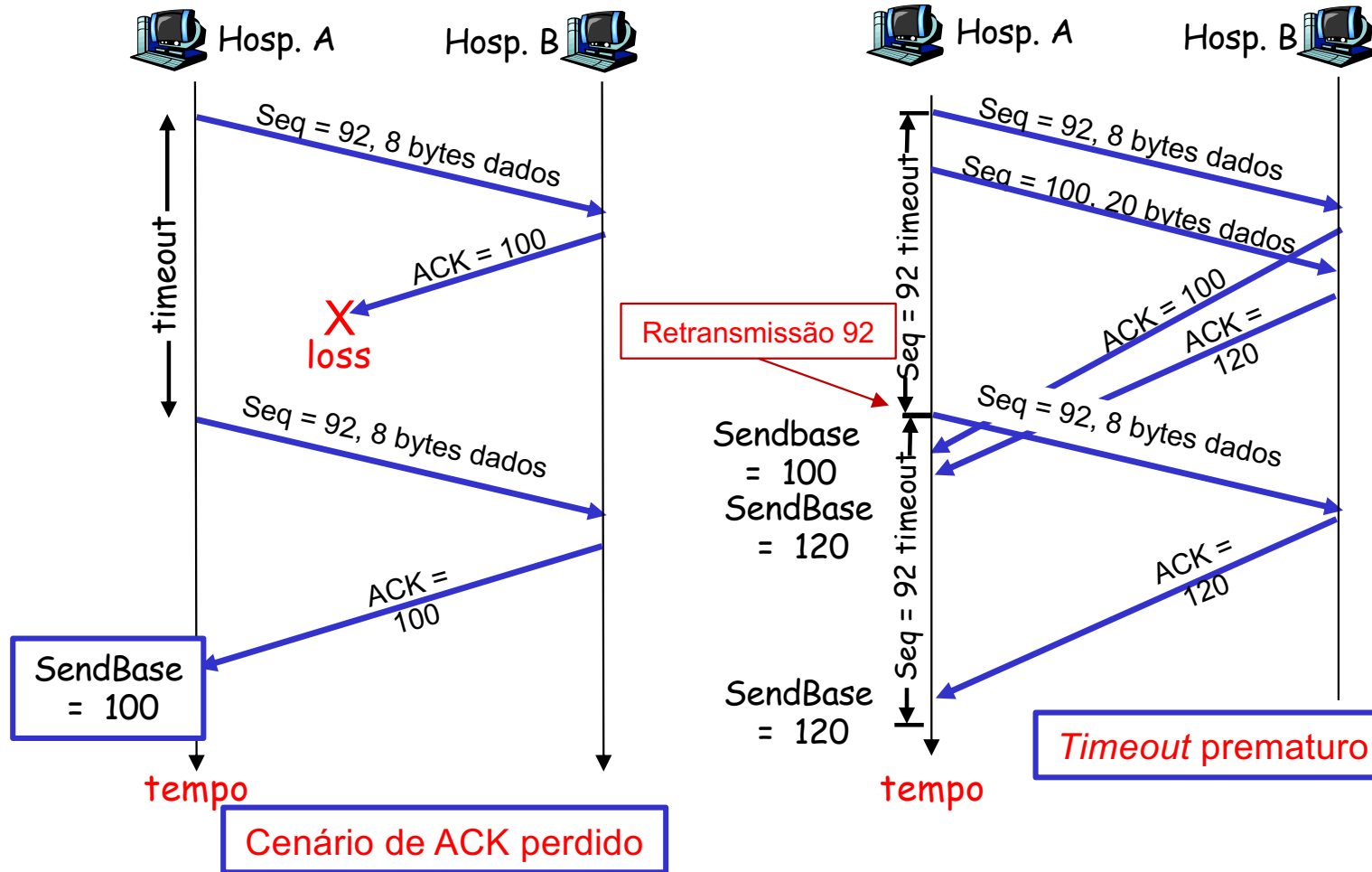
2.) ACK recebido:

- ❑ Confirma o seguimento corrente, e todos os demais segmentos anteriores ainda sem ACK.
 - Atualiza aqueles que sabidamente foram recebidos.
 - Receptor não enviaria um ACK se não tivesse enviado os anteriores.
- ❑ Reinicia *timer* (se houver segmentos pendentes)

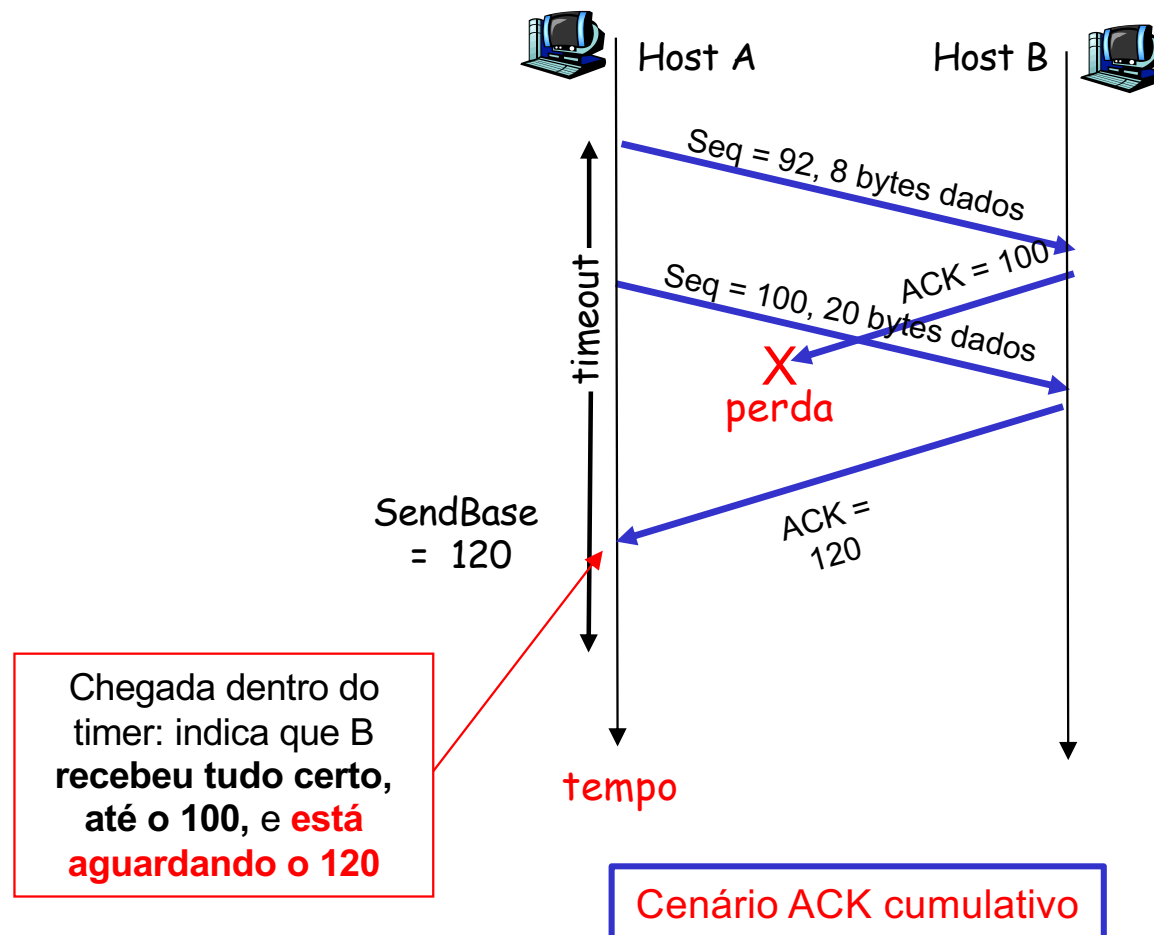
TCP: cenários de retransmissão (1)



TCP: cenários de retransmissão (2)



TCP: cenários de retransmissão (3)



Tratando segmentos faltantes: ACKs repetidos (1)

- ❑ Quando um **receptor** recebe um segmento com um **número de sequência maior do que próximo número de sequência esperado**, ele detecta uma **falha no fluxo** de dados
 - Detecta **segmento faltante**.
- ❑ TCP **não** usa NACK:
 - O **receptor não** pode emitir **não-ACK** para o emissor.
 - Ao invés disso: “**re-reconhece**” o **último byte** “**em ordem**” dos dados que ele recebeu corretamente.
 - Isto é, ele **gera um ACK em duplicata** para o último byte recebido corretamente.

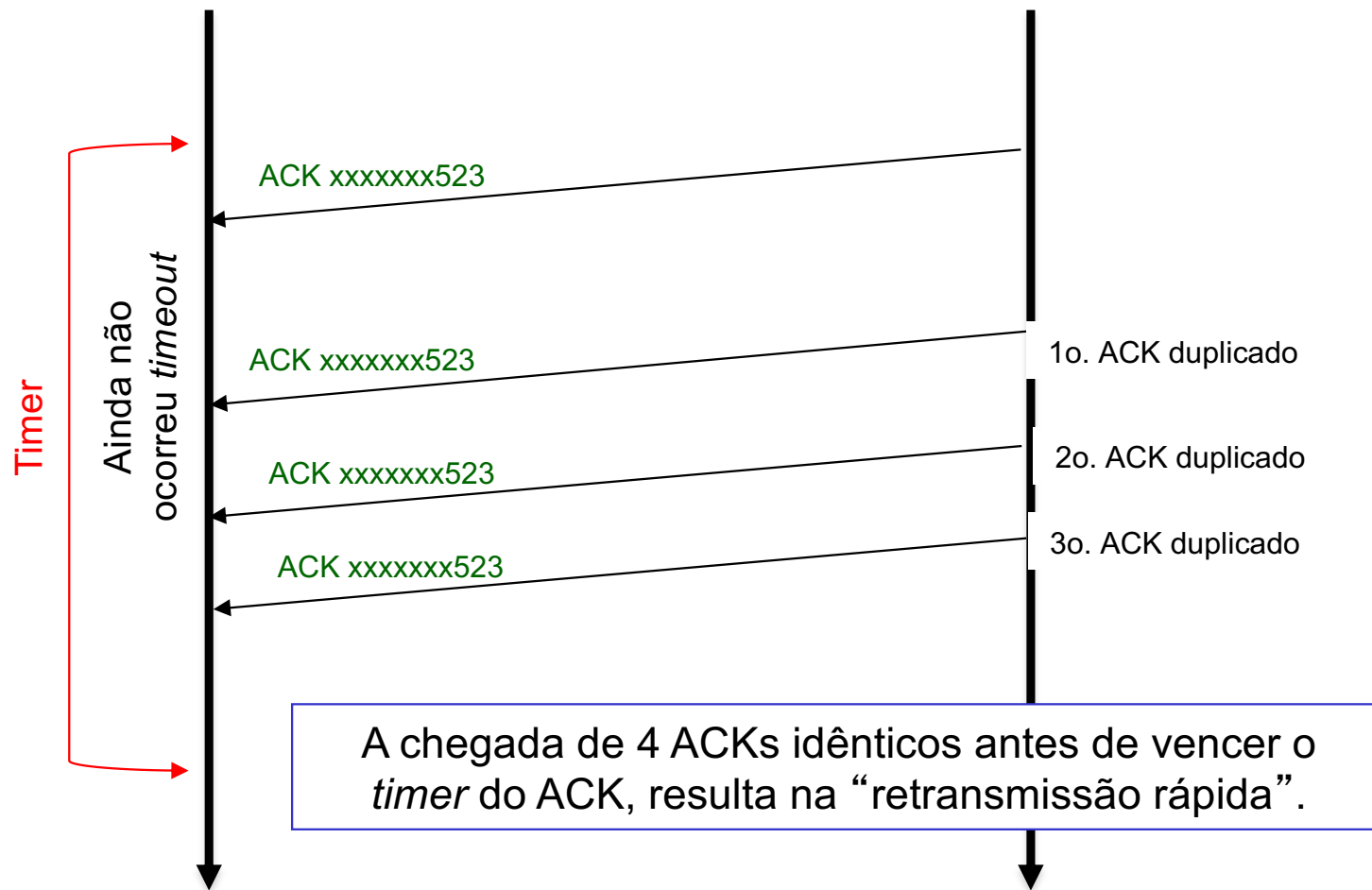
Tratando segmentos faltantes: ACKs repetidos (2)

- ❑ Período de *timeout* é relativamente grande:
 - Pode haver longo atraso antes de reenviar pacote perdido.
- ❑ É necessário detectar segmentos perdidos por meio de ACKs duplicados
 - Emissor geralmente envia muitos segmentos um após o outro.
 - Se um **segmento for perdido**, provavelmente haverá muitos ACKs duplicados para esse segmento.
 - Isso é tratado com a “retransmissão rápida”.

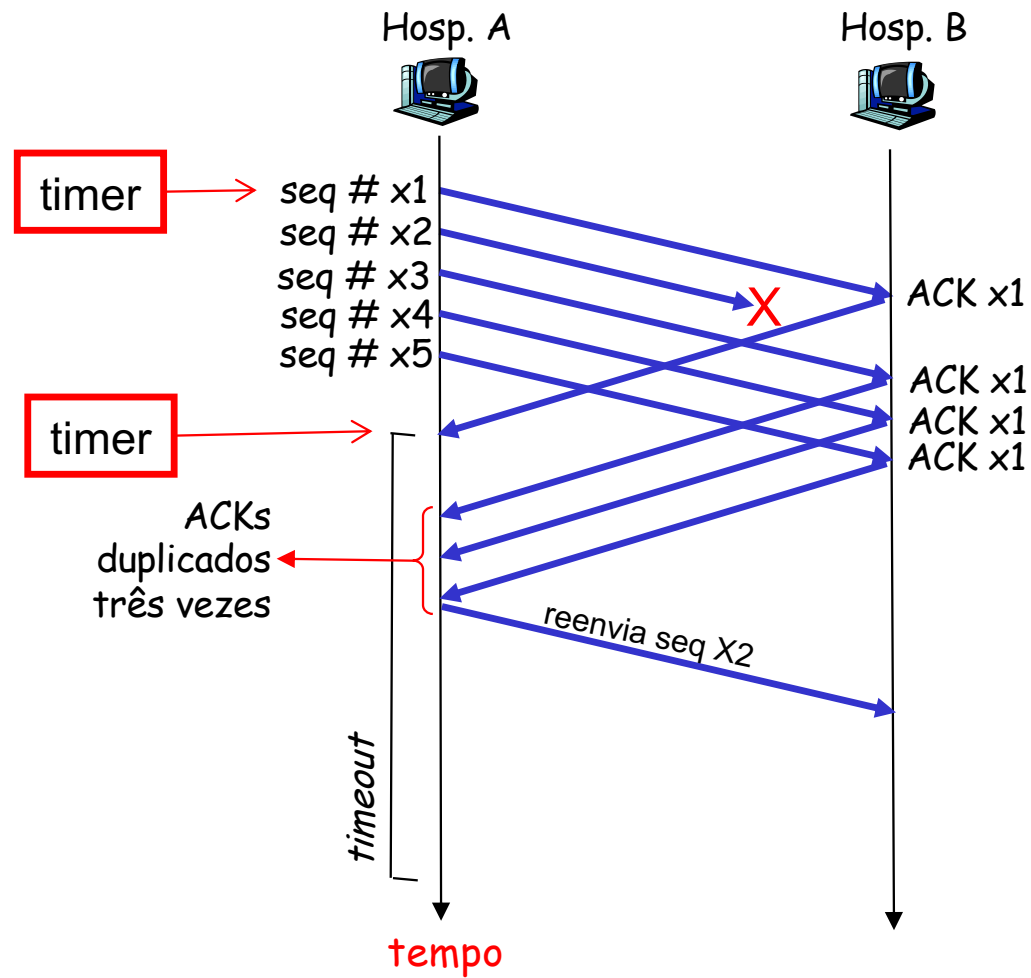
Retransmissão rápida (1)

- ❑ Se o **emissor** receber três **ACKs duplicados** (ou seja, **4 ACKs idênticos**) para o mesmo segmento que ele já enviou:
 - Assume que **foi perdido** o segmento **em seguida** ao segmento que foi confirmado 4X
 - O TCP executa **uma retransmissão rápida**.
 - Re-envia o segmento faltante, mesmo **antes** que o **temporizador esgote**
 - Definido pelo **RFC 2581**.

Entendendo os 3 “ACKs duplicados” (1)



Entendendo os 3 “ACKs duplicados” (2)



Algoritmo de retransmissão rápida:

```
event: ACK received, with ACK field value of y
    if (y > SendBase) {
        SendBase = y
        if (there are currently not-yet-acknowledged
segments)
            start timer
    }
    else {
        increment count of dup ACKs received for y
        if (count of dup ACKs received for y = 3) {
            resend segment with sequence number y
        }
    }
```

ACK duplicado para
segmento já com ACK

retransmissão rápida

RFC 2581 - TCP Congestion Control

*" The TCP sender SHOULD use the "fast retransmit" algorithm to detect and repair loss, based on incoming duplicate ACKs. The fast retransmit algorithm uses the arrival of 3 duplicate ACKs (**4 identical ACKs without the arrival of any other intervening packets**) as an indication that a segment has been lost. After receiving 3 duplicate ACKs, TCP performs a retransmission of what appears to be the missing segment, without waiting for the retransmission timer to expire."*

<http://www.faqs.org/rfcs/rfc2581.html>

Resposta de ACKs no TCP [RFCs 1122, 2581]

Evento	Ação do receptor TCP
Chegada do segmento em ordem, com número de seqüência previsto: todos os dados até o No. de seq. previsto já reconhecidos ; nenhum “buraco” nos dados recebidos.	Promove ACK atrasado → Espera até 500ms pela chegada de um outro segmento em ordem. Se o próximo segmento não chegar em ordem neste intervalo, emita um ACK do anterior .
Chegada de um segmento em ordem com número de seqüência previsto. Um outro segmento em ordem aguardando transmissão de ACK. Nenhum “buraco” nos dados recebidos.	Emita imediatamente um único ACK cumulativo. Confirmando ambos os segmentos em-ordem.
Chegada do segmento fora de ordem com número de seqüência mais alto do que esperado → detectada falha na seqüência .	Emita imediatamente o ACK duplicado , indicando o número de seqüência do próximo segmento esperado.
Chegada de segmento que completa parcial ou completamente uma falha nos dados recebidos.	Emita imediatamente o ACK , contanto que o segmento comece no fim mais baixo da falha.

Encerramento da Conexão

□ *Half-Close*

- Conexões TCP são *full-duplex*.

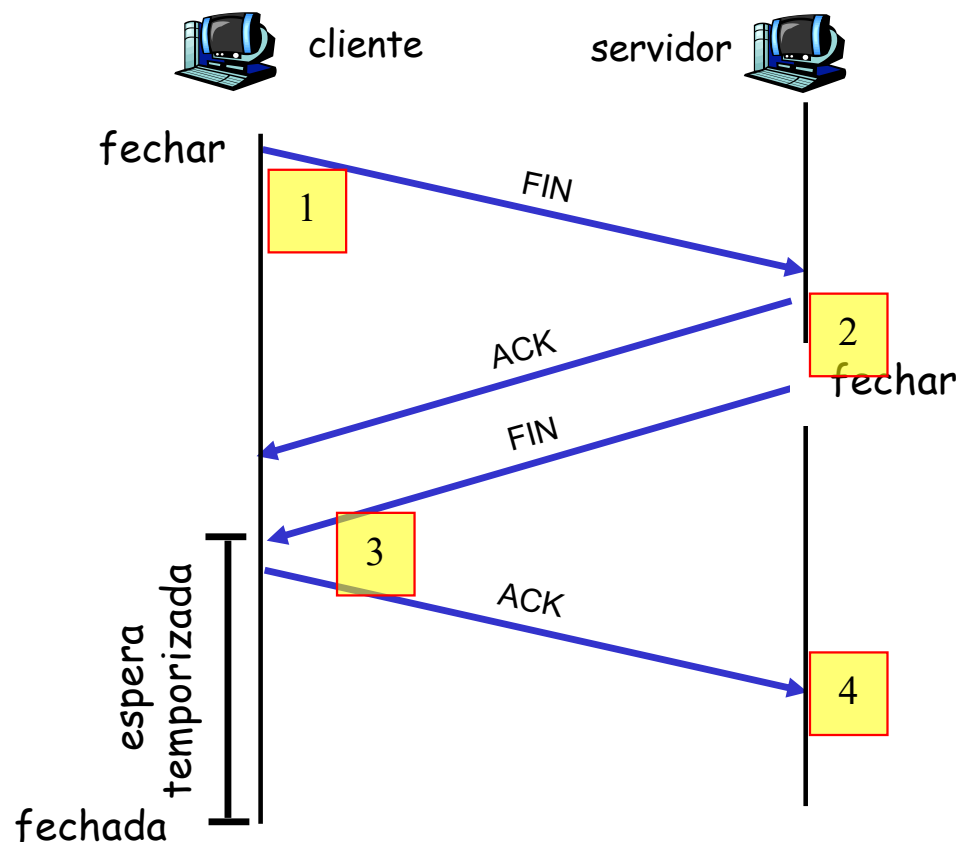
Cada lado da conexão deve finalizar a conexão de forma independente.

- Quando um dos lados envolvidos recebe uma solicitação de finalização, **deve enviar a notificação para a aplicação.**
 - Uma aplicação após receber o pedido de finalização ainda pode mandar dados.

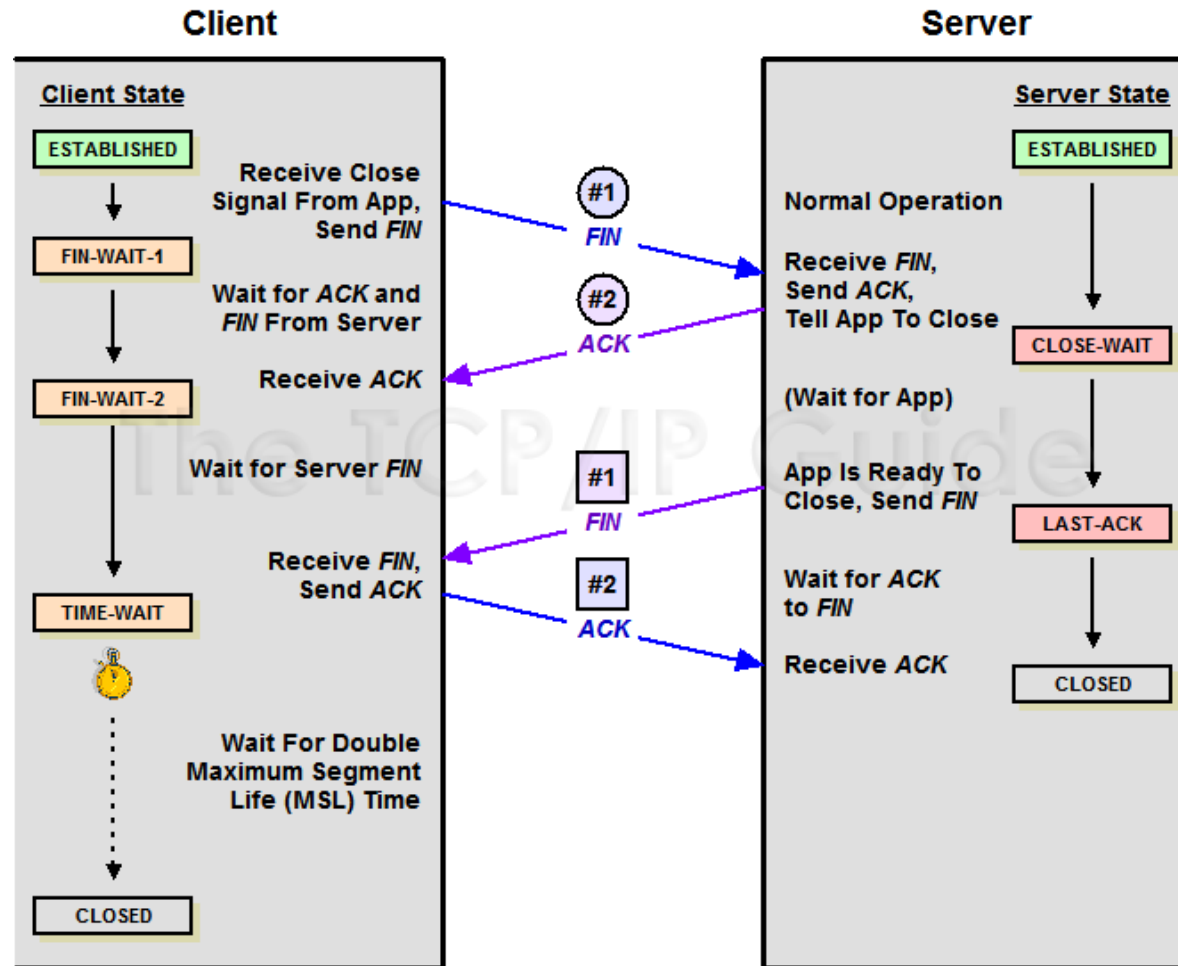
TCP: Fechando uma Conexão (“Half-close”)

Encerrando uma conexão:

- 1: **cliente** envia segmento de controle **FIN** ao servidor.
 - 2: **servidor** recebe FIN, responde com **ACK**. Também pede encerramento da conexão, enviando **FIN**.
 - 3: **cliente** recebe **FIN**, responde com **ACK**.
 - Entre em “espera temporizada” - responderá com ACK a FINs recebidos
 - 4: **servidor**, recebe **ACK**.
- Conexão encerrada.



Half-Close / Time-wait detalhado



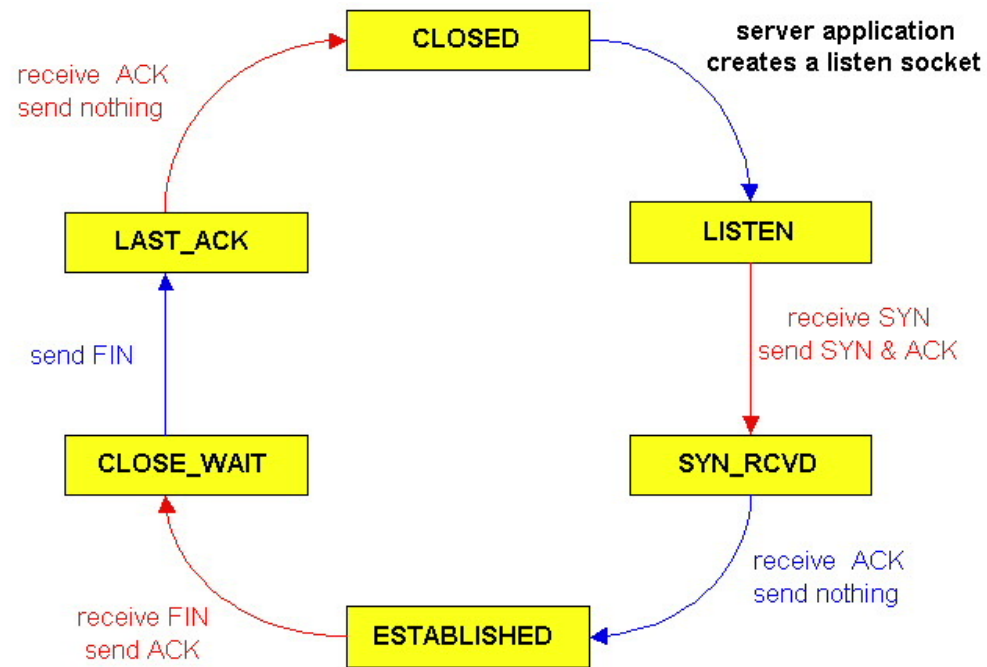
Half-close captura de pacotes:

```
svr4 % telnet bsdi.discard
Trying 192.82.148.3 ...
Connected to bsdi.
Escape character is '^]'.
^]                                #type Control, right bracket to talk to the Telnet
telnet> quit                      #terminate the connection
Connection closed.
```

Extraído de: TCP/IP Illustrated Vol.1 – Richard Stevens

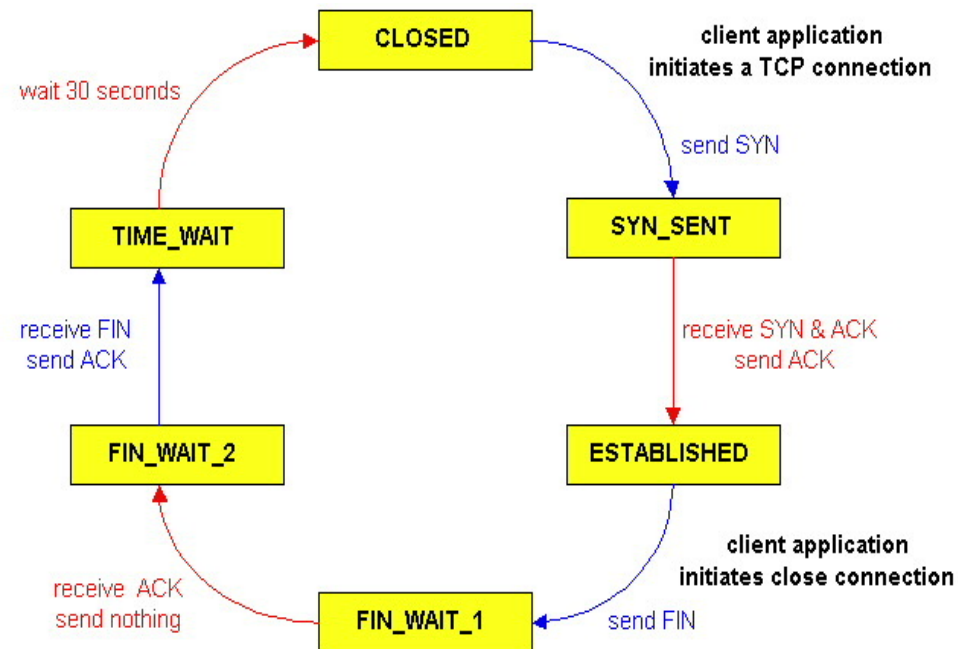
1	0.0	svr4.1037 > bsdi.discard: S 1415531521:1415531521(0) win 4096 <mss 1024>
2	0.002402 (0.0024)	bsdi.discard > svr4.1037: S 1823083521:1823083521(0) ack 1415531522 win 4096 <mss 1024>
3	0.007224 (0.0048)	svr4.1037 > bsdi.discard: . ack 1823083522 win 4096
4	4.155441 (4.1482)	svr4.1037 > bsdi.discard: F 1415531522:1415531522(0) ack 1823083522 win 4096
5	4.156747 (0.0013)	bsdi.discard > svr4.1037: . ack 1415531523 win 4096
6	4.158144 (0.0014)	bsdi.discard > svr4.1037: F 1823083522:1823083522(0) ack 1415531523 win 4096
7	4.180662 (0.0225)	svr4.1037 > bsdi.discard: . ack 1823083523 win 4096

TCP: Ciclo de vida do SERVIDOR



Exercício: estudar o ciclo de vida do **servidor** TCP

TCP: Ciclo de vida do CLIENTE



Exercício: estudar o ciclo de vida do **cliente** TCP

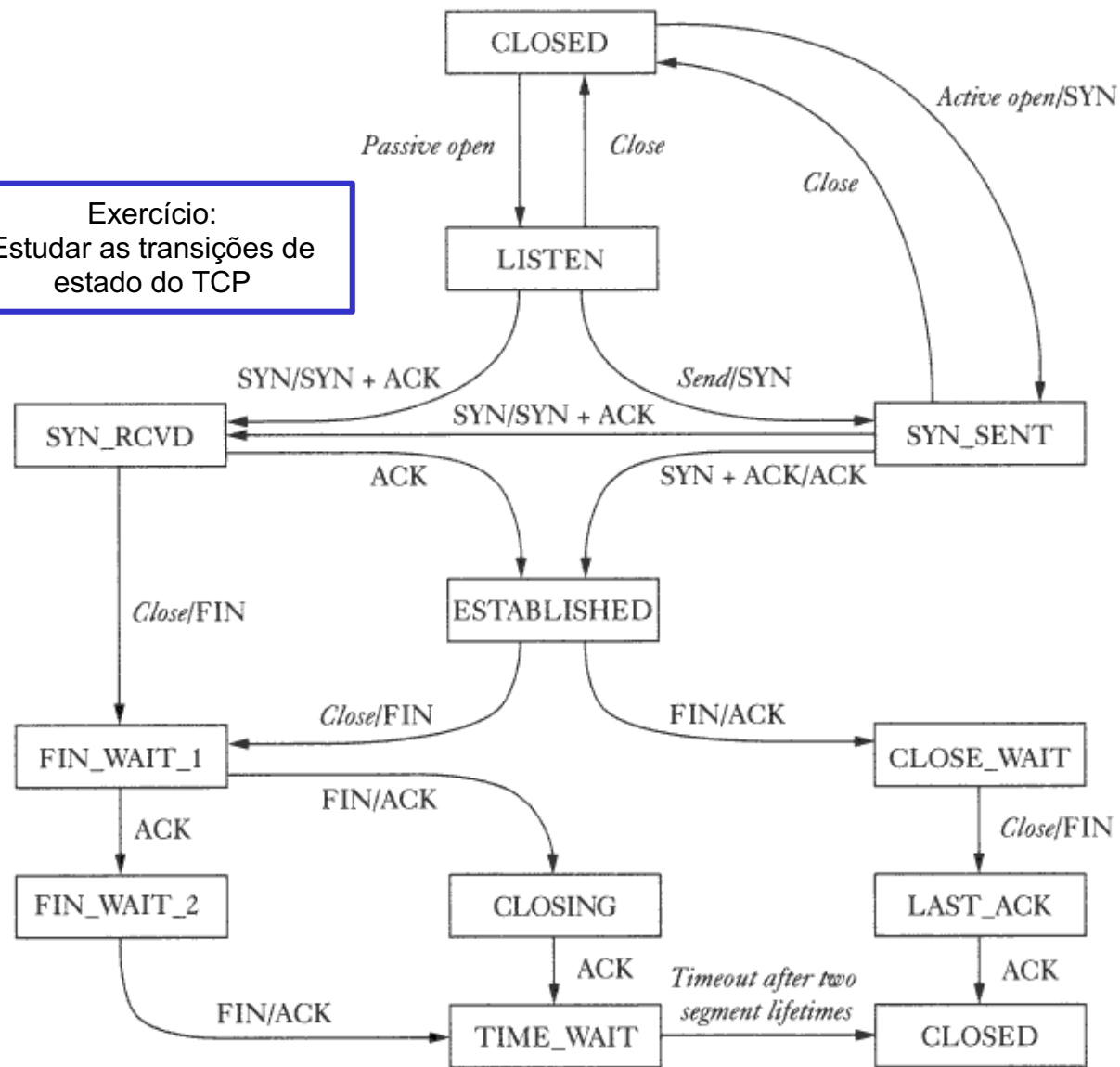
```
[adriano@Ono-Sendai ~ % netstat -a | more
```

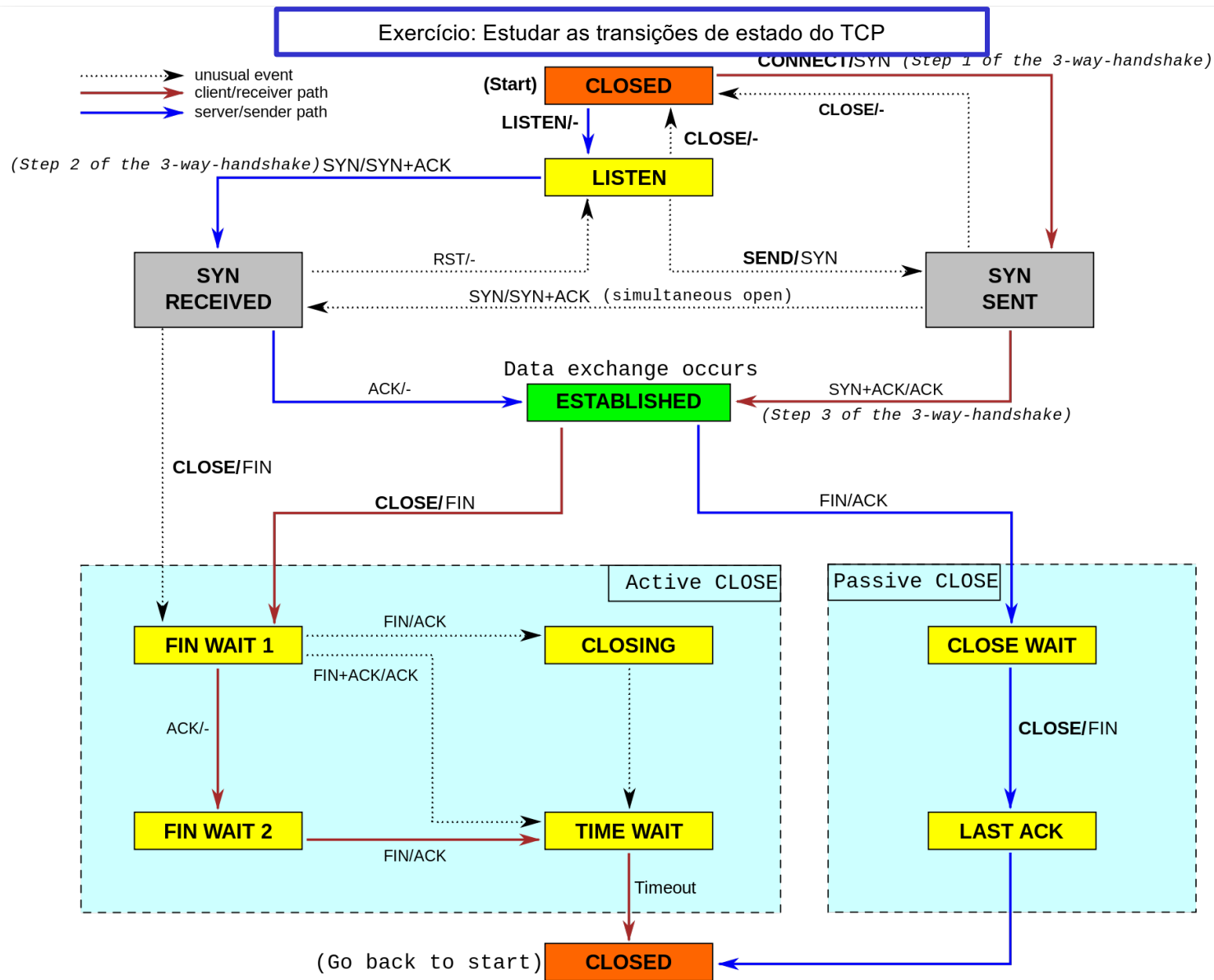
```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	0	ono-sendai.49773	android.googleap.https	ESTABLISHED
tcp4	0	0	ono-sendai.49770	presence.googlea.https	ESTABLISHED
tcp4	0	0	ono-sendai.49769	gru06s72-in-f10..https	ESTABLISHED
tcp4	0	0	ono-sendai.49768	gru06s71-in-f10..https	ESTABLISHED
tcp6	0	0	ono-sendai.local.cap	fe80::7a62:d757::1025	ESTABLISHED
tcp6	0	0	ono-sendai.local.1024	fe80::7a62:d757::1024	ESTABLISHED
tcp4	0	0	ono-sendai.49715	idrive.com.https	ESTABLISHED
tcp4	0	0	ono-sendai.49691	52.111.225.6.https	ESTABLISHED
tcp4	0	0	ono-sendai.49686	gru14s25-in-f10..https	ESTABLISHED
tcp4	0	0	ono-sendai.49589	52.226.139.185.https	ESTABLISHED
tcp4	0	0	ono-sendai.49553	youtubei.googlea.https	ESTABLISHED
tcp4	0	0	ono-sendai.49552	gru14s31-in-f10..https	CLOSE_WAIT
tcp4	0	0	ono-sendai.49466	gru14s30-in-f10..https	CLOSE_WAIT
tcp4	0	0	ono-sendai.49465	gru14s30-in-f10..https	CLOSE_WAIT
tcp4	0	0	ono-sendai.49464	gru14s30-in-f10..https	CLOSE_WAIT
tcp6	0	0	localhost.6263	*.*	LISTEN
tcp4	0	0	localhost.6263	*.*	LISTEN
tcp6	0	0	localhost.6258	*.*	LISTEN
tcp4	0	0	localhost.6258	*.*	LISTEN
tcp6	0	0	localhost.7679	*.*	LISTEN
tcp4	0	0	localhost.49152	localhost.49308	ESTABLISHED

```
:...skipping...
```


Exercício:
Estudar as transições de
estado do TCP





TCP: RTT e *timeout*

- Timer adaptativo.
- Outros *timers*.

RTT e *timeout* adaptativo do TCP (1)

Como definir o *timeout (RTO)*?

- ❑ Deveria ser **maior que RTT**.
 - Mas RTT varia bastante, dependendo do tráfego da rede.
- ❑ Se for muito **curto**: **retransmissões desnecessárias**.
- ❑ Se for muito **longo**: **demora para reagir a perdas**.

Como estimar o RTT? Vamos usar 3 variáveis:

- ❑ *EstimatedRTT* ou *SRTT* (RTT Estimado)
- ❑ *SampleRTT* ou *R* (RTT Amostrado ou RTT medido)
- ❑ *DevRTT* ou *RTTVAR* (RTT Desviado ou RTT variável)

RTT e *timeout* adaptativo do TCP (2)

❑ Variável ***SampleRTT (R)***:

- Tempo medido de uma AMOSTRA desde a transmissão, até receber o ACK.
- É feita uma amostragem a cada viagem de ida e volta.
- Então ***SampleRTT*** vai variar.

❑ Variável ***EstimatedRTT (SRTT)***:

- Média ponderada atual das medições.
- Desejável que o **EstimatedRTT** seja **o mais estável possível**.
- A ideia é usar a **média** de diversas **medições recentes**, e não apenas ***SampleRTT*** atual.

Primeira medição:

□ Inicialmente *timeout* (*RTO*) é ajustado para **1 seg**, enquanto não há medição.

□ Quando a primeira medição *R* do RTT é feita, o host DEVE definir:

$RTO \leftarrow 1;$

$SRTT \leftarrow R;$

$RTTVAR \leftarrow R/2;$

$RTO \leftarrow SRTT + K * RTTVAR;$

Onde $K = 4$ (determinado empiricamente)

Ver: <https://tools.ietf.org/html/rfc6298>

Segunda medição, e medições subsequentes:

- Quando uma medição RTT subsequente R' é feita, o host DEVE definir:

$$RTTVAR \leftarrow (1 - \beta) * RTTVAR + \beta * |SRTT - R'|;$$

$$SRTT \leftarrow (1 - \alpha) * SRTT + \alpha * R';$$

- O algoritmo acima DEVERIA ser calculado usando $\alpha = 1/8$ e $\beta = 1/4$ (*), ficando:

$$RTTVAR \leftarrow (1 - 0.25) * RTTVAR + 0.25 * |SRTT - R'|;$$

$$SRTT \leftarrow (1 - 0.125) * SRTT + 0.125 * R';$$

Após o cálculo, um host DEVE atualizar:

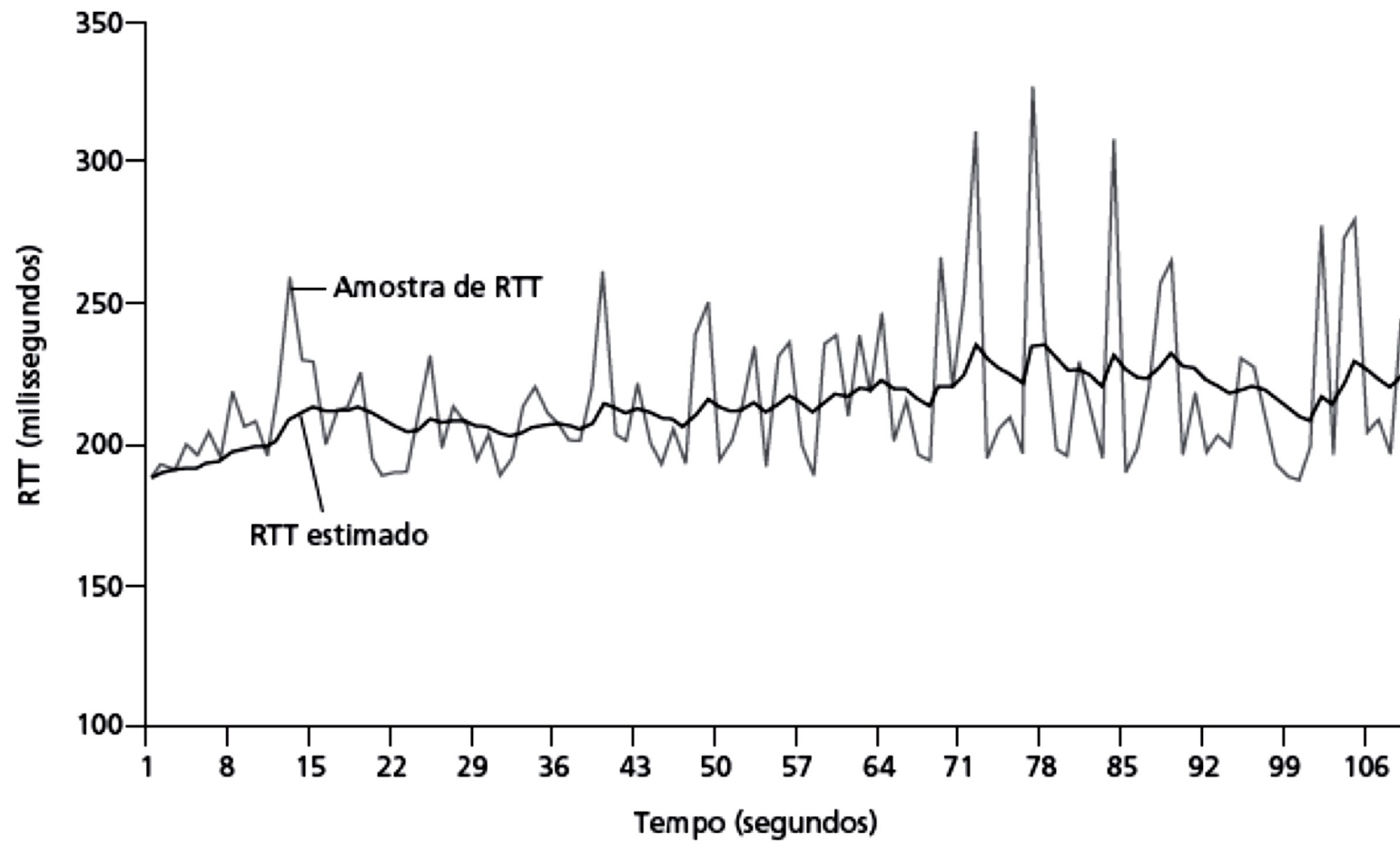
$$RTO \leftarrow SRTT + K * RTTVAR; (**)$$

Onde $K = 4$. (definido empiricamente)

(*) conforme sugerido em [JK88]

(**) ver exercício no slide adiante ("Clock Granularity")

Relação entre RTT amostra e RTT estimado



RTT e *timeout* adaptativo do TCP (4)

<https://tools.ietf.org/html/rfc6298>

Na verdade, pelo [RFC 6298](#) o Timeout é calculado da seguinte forma:

$$RTO \leftarrow SRTT + \max(G, K \cdot RTTVAR);$$

Seu exercício aqui é entender o parâmetro G , chamado no RFC 6298 de “**Clock Granularity**”.

Observe que [[Jac88](#)] e [[JK88](#)] descrevem vários truques inteligentes que podem ser usados para obter melhor precisão de temporizadores de granularidade grosseira. Essas mudanças são amplamente implementadas nas implementações atuais do TCP.

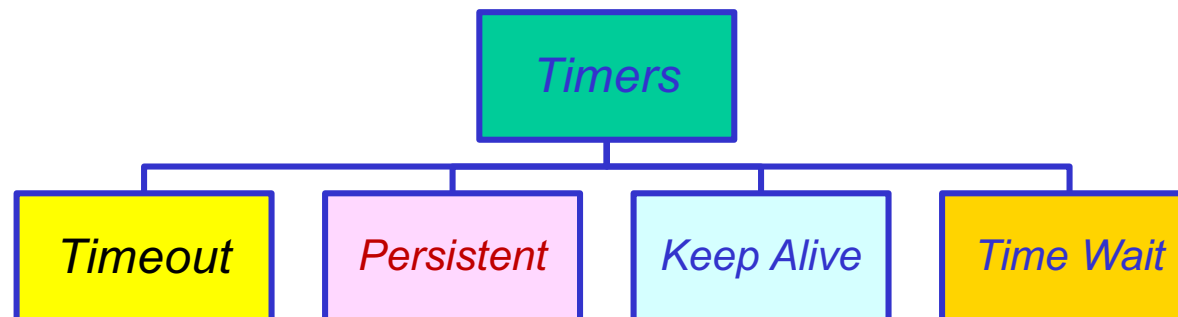
Consulte:

- [[JK88](#)] Jacobson, V. and M. Karels, "Congestion Avoidance and Control", <ftp://ftp.ee.lbl.gov/papers/congavoid.ps.Z>.
- [[Jac88](#)] Jacobson, V., "Congestion Avoidance and Control", Computer Communication Review, vol. 18, no. 4, pp. 314-329, Aug. 1988.

Exercício: Pesquisar e estudar algoritmo **Jacobson / Karels**.

Se você pensa que acabou... (1)

- ❑ Temporizador de retransmissão não é o único.
- ❑ Na verdade são 4 *timers*.
 1. **Timeout adaptativo** (visto)
 2. ***Timer de Persistência.***
 3. ***Timer Keep-alive.***
 4. ***Timer Time-wait.***



1. Timer de persistência

□ É usado para **evitar impasse**.

- **Receptor** envia pacote de **tamanho de janela 0**, pedindo para emissor **aguardar**.

(Mais adiante ficará mais claro o que significa um “**tamanho de janela zero**”)

- **Emissor** faz um teste de tempos em tempos, usando o timer de persistência, para ver se pode voltar a transmitir (**devido a risco de perda do pacote de atualização do receptor**).

2. Timer *keep alive*

- ❑ “**Mantenha Vivo**”:
- ❑ Para verificar conexões inativas.
- ❑ Um lado verifica se o outro ainda está lá.

Veremos mais adiante como isso será feito, usando o **tamanho da janela**.

3. Temporizador *Time Wait*

❑ Temporizador *time wait* (“tempo de espera”):

- Usado para **encerramento** de sessão.
- É ajustado para 2 vezes o **tempo de vida máximo de um pacote** (TTL - *Time to Live*).
- Tenta garantir que quando uma sessão for encerrada, todos os pacotes criados por ela já tenham sido entregues.

Controle de FLUXO no TCP

TCP: Controle de Fluxo (1)

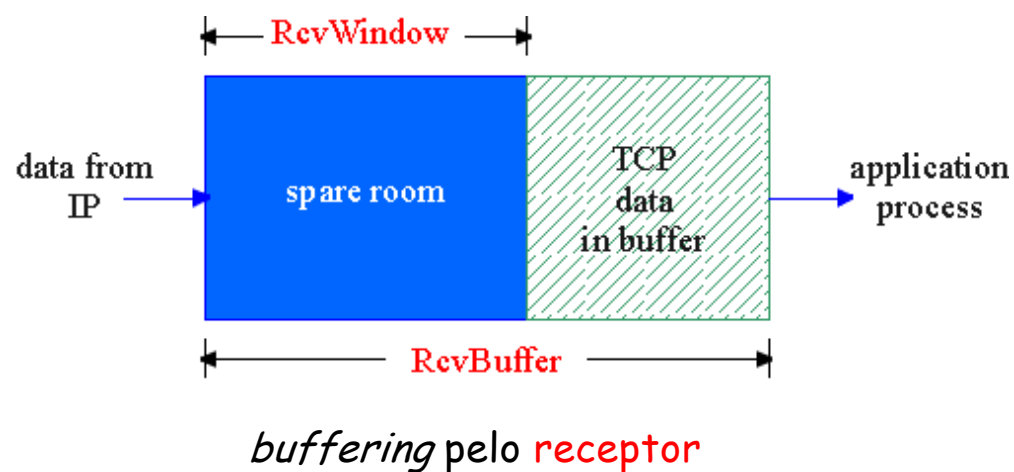
Para **emissor** não esgotar os *buffers* do **receptor**, transmitindo demais, ou muito rapidamente.

Serão necessárias mais 2 quantidades:

RcvBuffer = tamanho do Buffer de **recepção**.

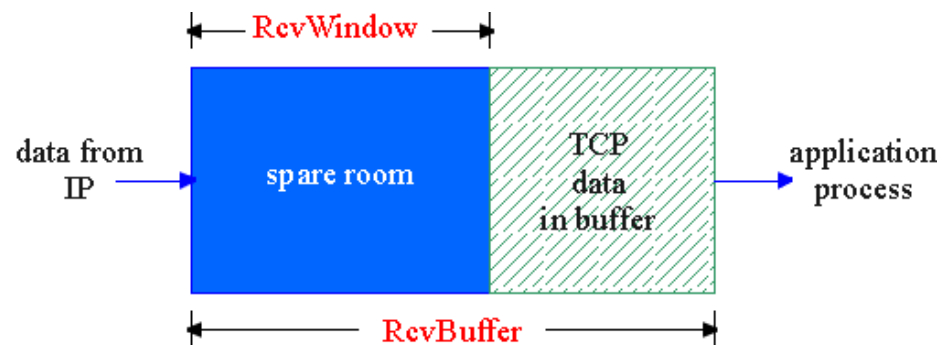
RcvWindow = informa ao **emissor** quantidade de espaço vazio no *buffer* do **receptor**.

Este valor é mantido como **variável** em **cada lado da conexão** (lembrar que é *Full Duplex*).



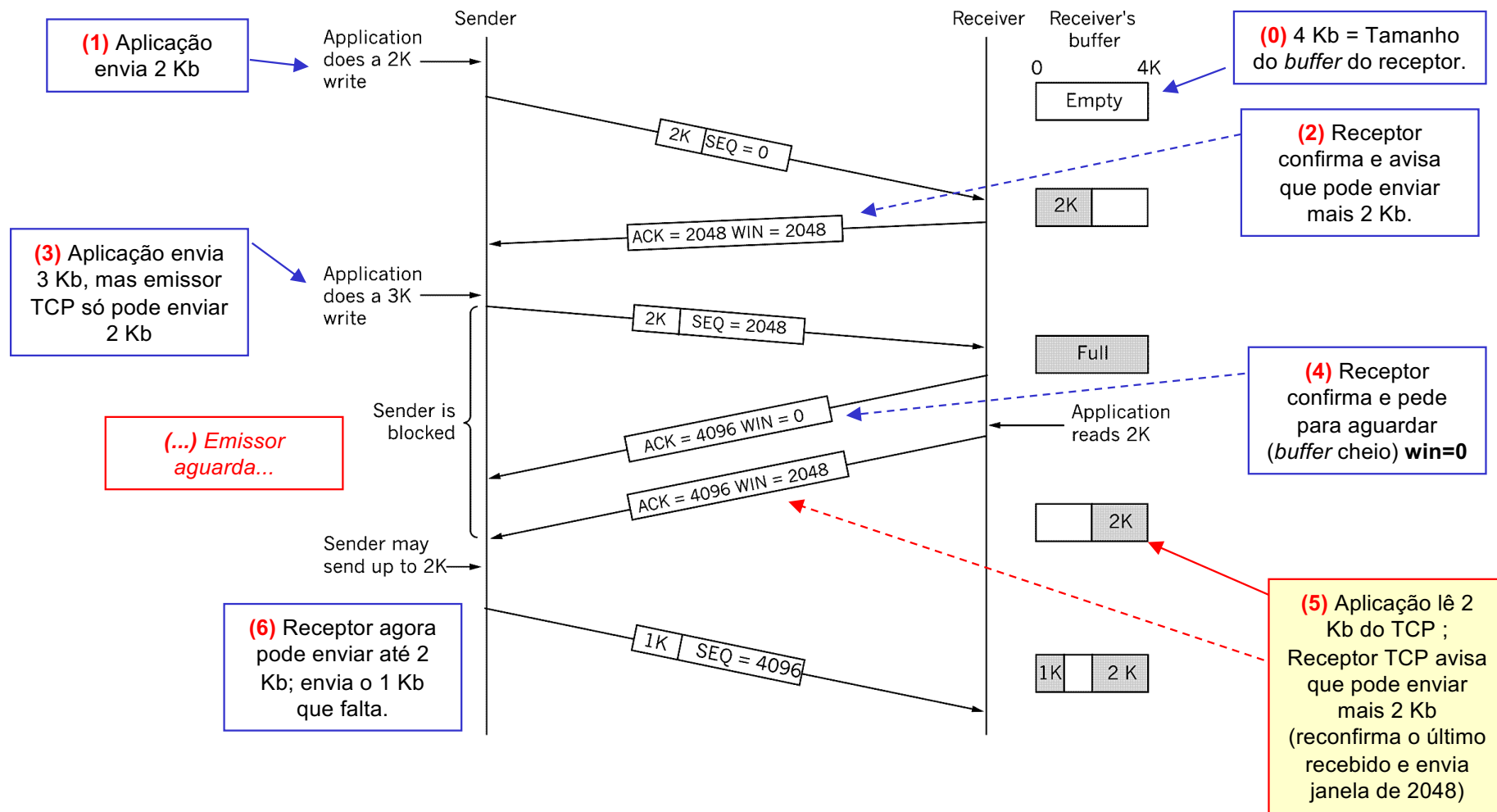
TCP: Controle de Fluxo (2)

- ❑ **RECEPTOR:** explicitamente avisa o **emissor** da quantidade de **espaço livre disponível** (que muda dinamicamente).
 - Campo **RcvWindow (janela)** no segmento TCP .
- ❑ **EMISSOR:** mantém a **quantidade de dados transmitidos, porém ainda não reconhecidos, MENOR** que o **valor mais recente de RcvWindow**.
(para cada lado da conexão)



0																16																31															
16-bit source port number																16-bit destination port number																															
32-bit sequence number																																															
32-bit acknowledgment number																																															
4-bit header length				Reserved (6 bits)				U R G	A C K	P S H	R S T	S S Y	F I N	16-bit window size																																	
16-bit TCP checksum																16-bit urgent pointer																															
Options (if any)																																															
Data (if any)																																															

Troca de informações sobre controle de fluxo TCP



Controle de Congestionamento

Princípios de Controle de Congestionamento (1)

Congestionamento:

- ❑ Trata **emissor** enviando dados mais rapidamente do que a **rede** pode suportar.
 - **Atenção!** É diferente de controle de fluxo.
 - **Retransmissão** de pacotes trata o sintoma do congestionamento da rede: **perda de segmentos**.
 - Mas **não trata a causa do congestionamento**:
 - Origens tentando enviar dados **numa taxa muito alta**, na qual a **rede** não está suportando.

Princípios de Controle de Congestionamento (2)

□ Como se manifesta:

- Perda (*drop*) de pacotes
 - Esgotamento de *buffers* em roteadores.
- Retransmissão de pacotes.
 - Devido aos *drops*.
- Longos atrasos
 - Grande filas nos *buffers* dos roteadores.

Solução para o Congestionamento

- ❑ Solução para o congestionamento da rede: **reduzir a taxa de transmissão de dados.**

Não inserir novos pacotes na rede, até que os antigos tenham saído.

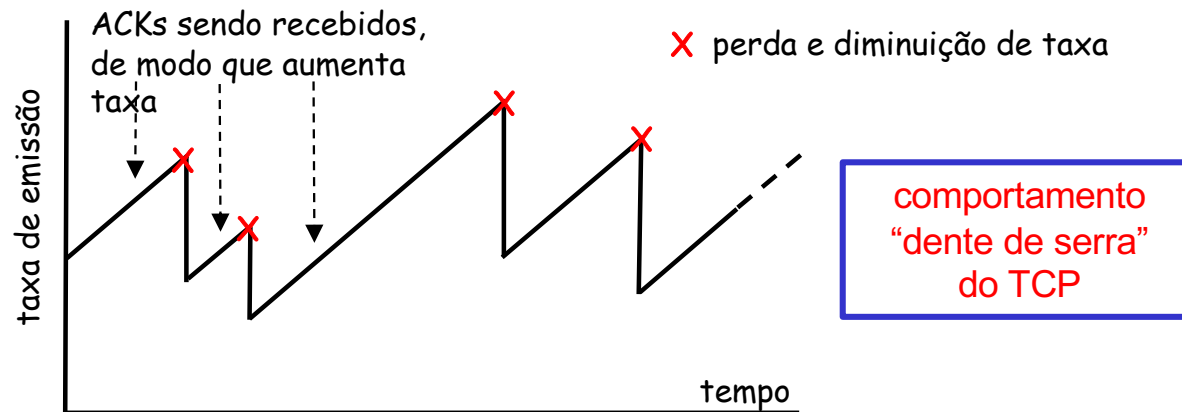
- Ou seja, até que os antigos **tenham sido entregues.**

- ❑ TCP → **tenta alcançar esse objetivo.**

Manipulando dinamicamente o tamanho da janela.

Controle de congestionamento TCP: busca por largura de banda adequada

- ❑ **Princípio da busca por largura de banda ótima:**
 - ❑ **Aumenta** taxa de transmissão no recebimento do ACK.
 - ❑ Até ocorrer perda.
 - ❑ Depois **diminui** taxa de transmissão, **quando ocorre perda**.
 - ❑ Aumentar com **ACK ok**, e diminuir na perda.



- ❑ Com que velocidade aumentar/diminuir? Veremos detalhes a seguir

O que o TCP faz para evitar congestionamentos:

- ❑ O **receptor** especifica uma **janela** a partir do **tamanho de seu *buffer***.
 - Indica o quanto ele está disposto a receber.
- ❑ Se o **transmissor** se mantiver **dentro do tamanho da janela**, **não haverá problemas causados pela sobrecarga dos *buffers* no receptor**.
 - Isso é o controle de fluxo (já visto).
 - Mas eles ainda podem ocorrer devido a **congestionamentos internos na rede**.

TCP: Controle de Congestionamento (1)

- Existem **dois** problemas potenciais:

- Capacidade do **receptor**.
- Capacidade da **rede**.

- Deve-se tratar cada um em separado.

- Para isso, cada **transmissor** mantém duas variáveis de janelas:

- **Janela fornecida pelo receptor.**

$RcvWin$ → recebido no campo do segmento TCP.

- **Janela de congestionamento:**

$CongWin$ → dinâmica.

 Vejamos

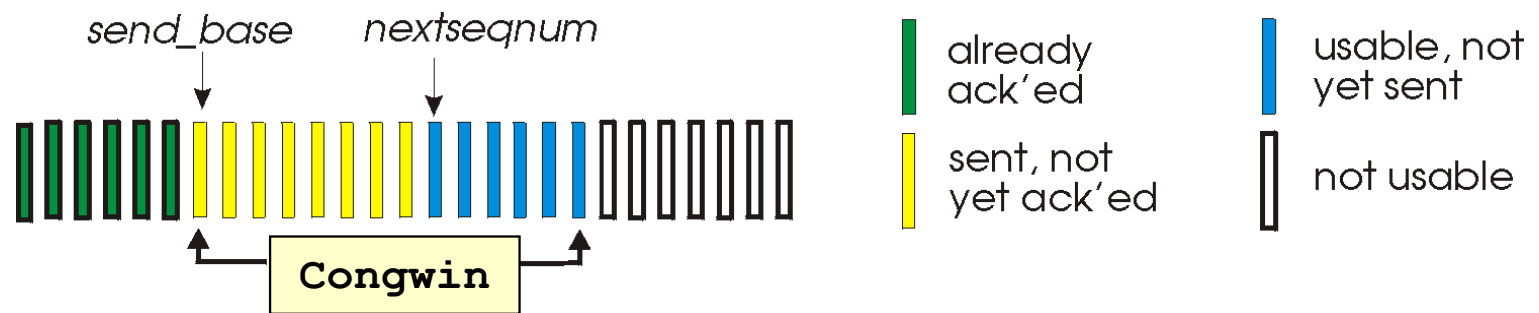
TCP: Controle de Congestionamento (2)

- ❑ Cada variável identifica o **número de bytes** que o **transmissor pode enviar**.
 - Pode enviar o valor mínimo das duas janelas.
- ❑ A quantidade máxima de dados **não confirmados** que um *host* pode enviar em uma conexão deve ser:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min\{\text{CongWin}, \text{RcvWin}\}$$

TCP: Controle de Congestionamento

- ❑ Controle é fim-a-fim (**sem feedback da rede**).
- ❑ Taxa de transmissão é limitada pelo tamanho da janela de congestionamento **Congwin**:



A janela efetiva é o mínimo do que o transmissor e o receptor consideram viável. Veremos exemplo a seguir....

TCP: Controle de Congestionamento

- A janela efetiva é o **mínimo** do que o **transmissor** e o **receptor** consideram viável.

Exemplos:

1. Se o **receptor** disser: “envie 8KB”, mas...
 - se o **transmissor** souber que qualquer rajada com mais de 4 KB poderá congestionar a rede,
 - **ele enviará apenas 4 KB.**
2. Se o **receptor** disser: “envie 8KB”, mas...
 - Se o **transmissor** souber que rajadas de até 32 KB passam pela rede sem problemas,
 - ele enviará os 8 KB solicitados.

TCP: Controle de Congestionamento

- A janela efetiva é o **mínimo** do que o **transmissor** e o **receptor** consideram viável.

Exemplos:

1. Se o **receptor** disser: “envie 8KB”, mas...
 - se o **transmissor** souber que qualquer rajada com mais de 4 KB poderá congestionar a rede,
 - ele enviará apenas 4 KB.
2. Se o **receptor** disser: “envie 8KB”, mas...
 - Se o **transmissor** souber que rajadas de até 32 KB passam pela rede sem problemas,
 - ele enviará os 8 KB solicitados.

Como funciona o controle:

- ❑ **Sondagem:** Transmitir o mais rápido possível sem perder pacotes (ou seja, usar **Congwin** ajustado ao máximo possível)
 - Vai **Aumentar Congwin** até perder pacotes → vai **causar congestionamento**.
 - Quando houver perdas: **diminuir Congwin**.
 - Depois volta a à sondagem, aumentando novamente.
- ❑ Duas “fases”
 1. **Partida lenta**.
 2. **Prevenção de congestionamento**.
- ❑ Variáveis importantes:
 - **congrwin**.
 - **threshold**:
 - define **limiar** entre fases de partida lenta e controle de congestionamento.
 - Também chamado de “patamar”.

Tudo explicado a seguir..

Partida lenta e sondagem do congestionamento (1)

□ Uma vez que a conexão é estabelecida (pós-handshake):

- Transmissor ajusta *congwin* igual a 1 **MSS**.
 - Envia este segmento.
 - Se esse **segmento for confirmado antes de um *timeout***, então o **transmissor**:
 - **Adiciona +1 MSS na janela de congestionamento**, de modo que agora ela **equivale a 2 MSS**.
 - Ou seja: *congwin++*
 - Envia os 2 segmentos...
- (continua...)

Partida lenta e sondagem do congestionamento (2)

- ❑ À medida que cada um desses segmentos for confirmado, **a janela de congestionamento é aumentada em um tamanho** deste segmento máximo, de tal forma que - quando os 2 forem confirmados - a janela terá agora **4x** o valor inicial.
 - Quando a janela de congestionamento chegar a **n segmentos**, e se **todos os n segmentos forem confirmados** a tempo, **a janela de congestionamento será aumentada pelo número de bytes correspondentes aos n segmentos.**
- ❑ **Cada rajada confirmada duplica a janela de congestionamento atual: crescimento exponencial.**
- ❑ **Isso é chamado de “partida lenta” (*slow start*).**
 - (Na verdade a “partida lenta” não é tão lenta assim)
- ❑ Mas, e quando parar? Veremos adiante...

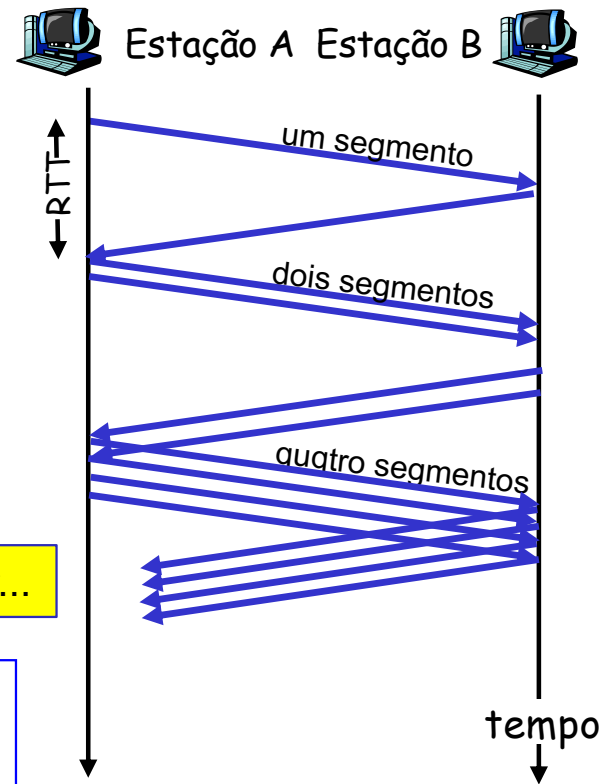
TCP: Partida lenta

Algoritmo de Partida Lenta

```
inicializa: Congwin = 1  
for (cada segmento c/ ACK)  
    Congwin++  
until (evento de perda OR  
        CongWin > threshold)
```

Veremos o que é isso a seguir...

- ❑ A cada RTT ocorre um **aumento exponencial** no tamanho da janela.
 - Partida **não é tão lenta!**



Quando parar o crescimento?

- ❑ Obviamente → este crescimento **terá de ser interrompido em algum momento.**
 - Devido a congestionamento da rede ou esgotamento da capacidade do receptor.
- ❑ Para isso: variável *threshold*.
- ❑ **Vejamos como...**

Threshold

- ❑ Terceiro parâmetro limitante: *threshold*.
- O **crescimento** exponencial da partida lenta é **interrompido** quando o ***threshold*** é atingido.
 - A partir de então, o crescimento passa a ser **linear**.
 - Quando atingir o *threshold*, a variável ***congwin*** passa a aumentar de 1 MSS **para cada envio**.
- ❑ *threshold*: definido inicialmente, por padrão, como **64KB**.

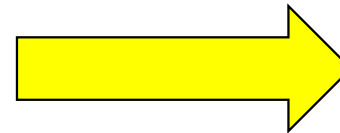
Threshold e o timeout

□ Quando ocorre um *timeout*:

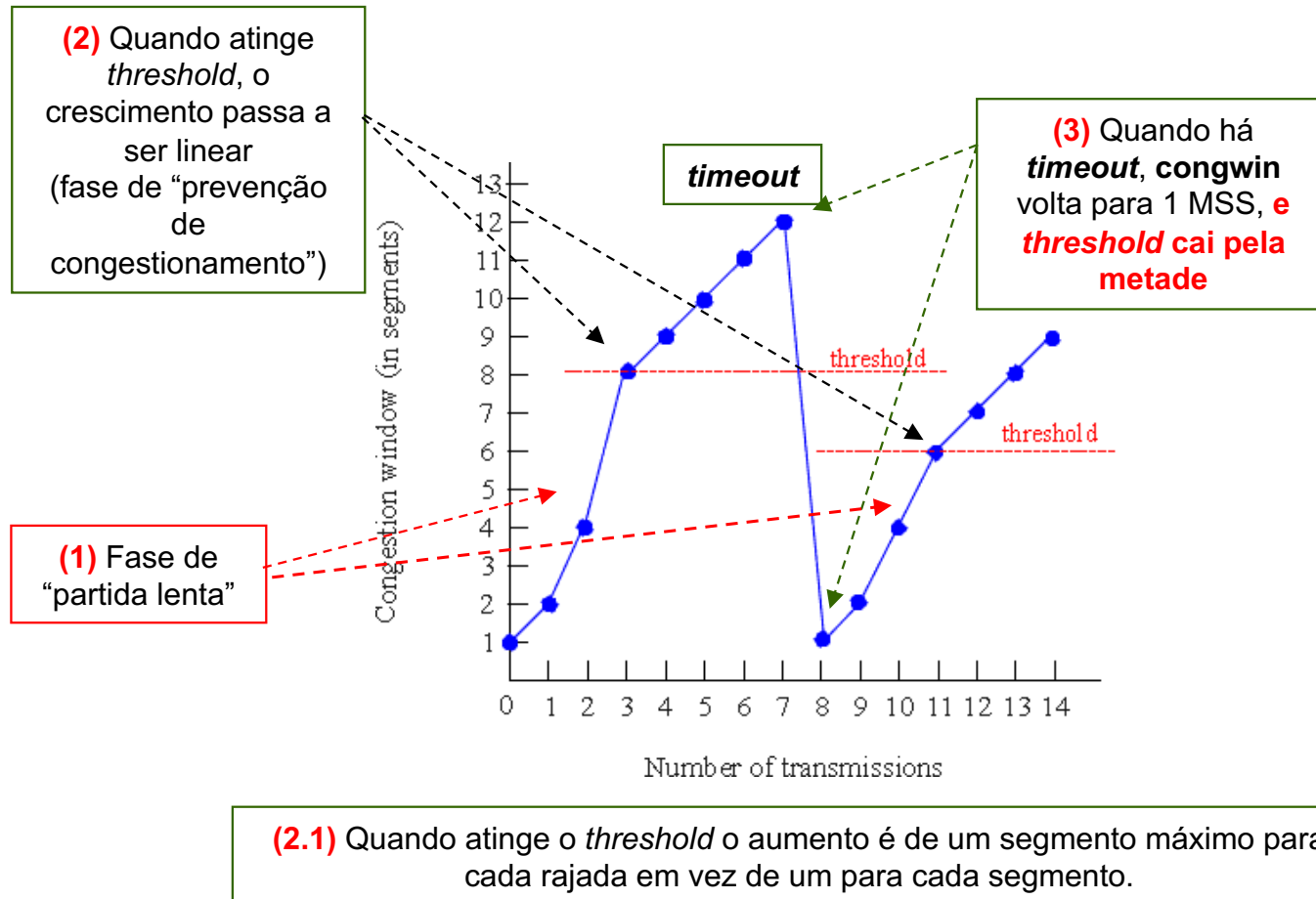
- O *threshold* é atribuído à metade da janela de congestionamento atual,
- e a janela de congestionamento é reiniciada para 1 MSS.

O algoritmo reduz o tamanho da janela de congestionamento à metade, e depois retoma seu crescimento a partir daí.

(Ficará mais fácil de entender no gráfico a seguir)



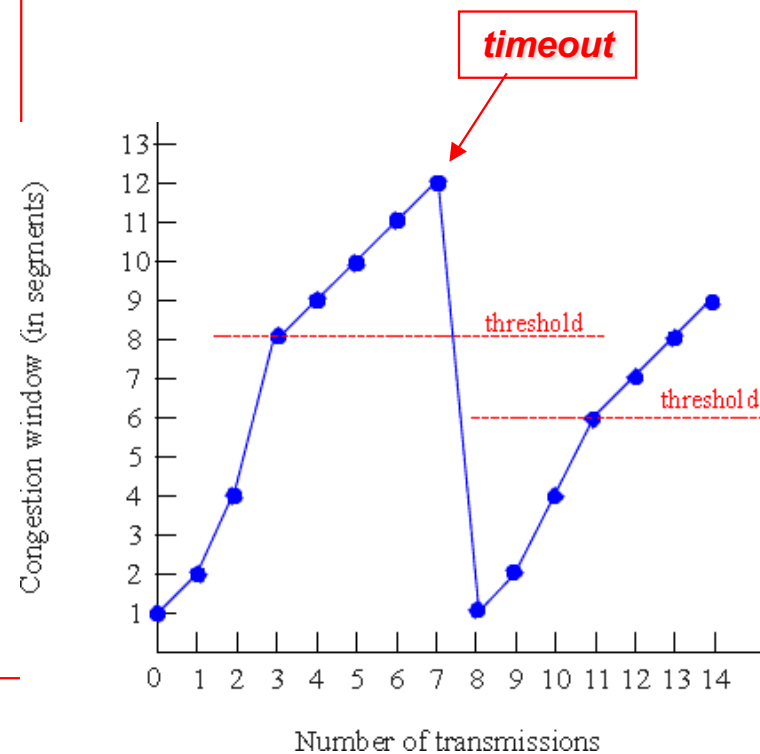
TCP: Evitar Congestionamento (1)



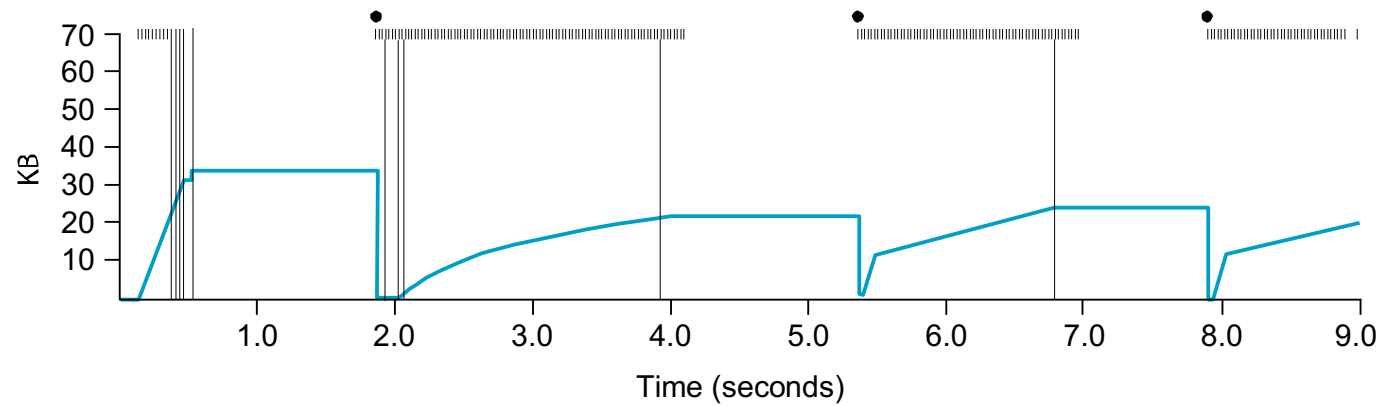
TCP: Evitar Congestionamento (2)

Evitar congestionamento

```
/* partida lenta acabou */  
/* Congwin > threshold */  
Until (event de perda) {  
    cada  $W$  segmentos  
    reconhecidos:  
        Congwin++  
}  
threshold = Congwin/2  
Congwin = 1  
faça partida lenta.
```



Comportamento do *TCP Congestion Control*



Exercício – TCP Reno x TCP Tahoe:

- ❑ Existem diferentes implementações de controle de congestionamento.
 1. Estudar a diferença entre “*TCP Reno*” e “*TCP Tahoe*”.
 2. Entender a diferença entre **retransmissão rápida** (*Fast Retransmit*) e **recuperação rápida** (*Fast Recovery*).
 - Responda: como isso impacta o controle de congestionamento?

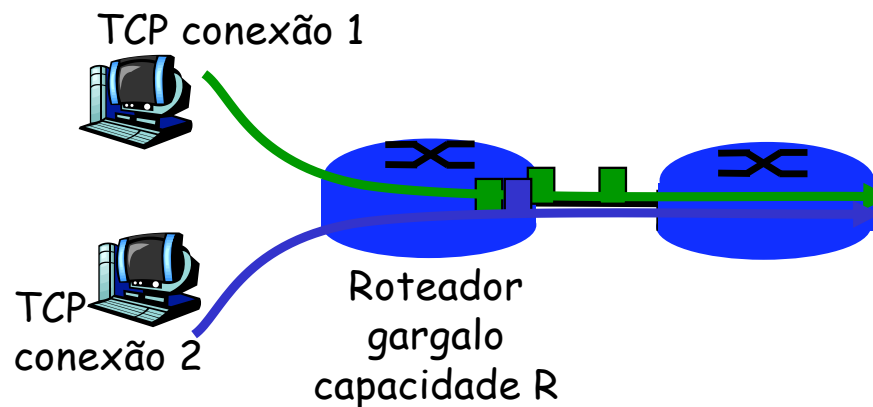
AADM / Additive-Increase, Multiplicative-Decrease (AIMD)

Justeza do TCP

- ❑ Desconsiderando a fase de partida lenta: TCP usa “AADM”:

- *Aumento Aditivo, Decremento Multiplicativo*
- **Aumenta** janela em 1 a cada RTT.
- **Diminui** janela por fator de 2 quando um evento de perda acontece.

Meta de justeza: se N sessões TCP compartilham o mesmo enlace, cada uma deve ganhar $1/N$ da capacidade do enlace.



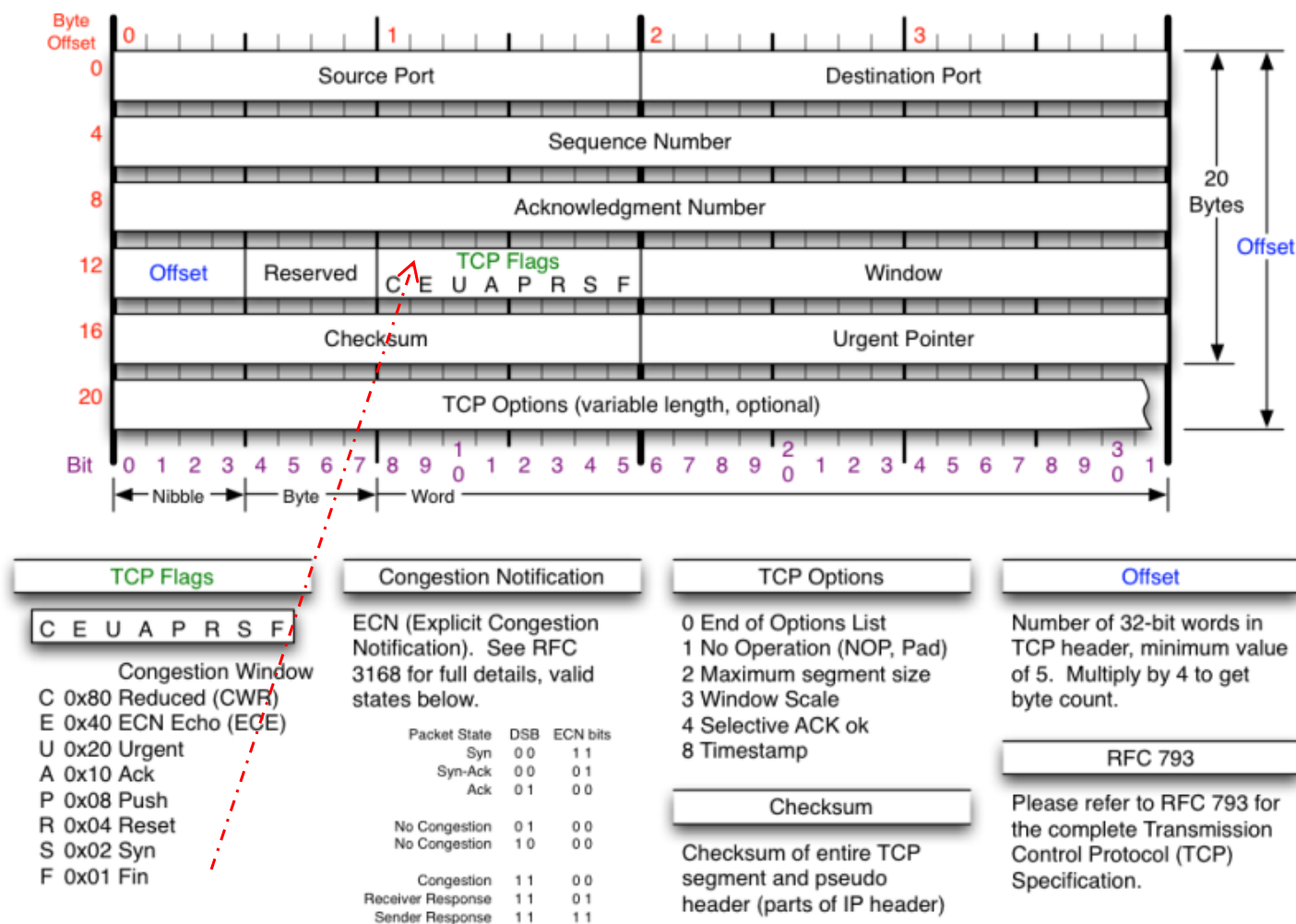
ECN - Explicit Congestion Notification flags (RFC-3168)

TCP Header																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																												
Offsets	Octet	0								1								2								3																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											
0	0	Source port																Destination port																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																										
4	32	Sequence number																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																										
8	64	Acknowledgment number (if ACK set)																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																										
12	96	Data offset			Reserved 0 0 0			N S	C	E	U	A	P	R	S	F	Window Size																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											
																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																												</

- **NS** (1 bit): ECN-nonce concealment protection (*) **experimental: see RFC 3540.** <https://www.icir.org/floyd/ecn.html>
- **CWR** (1 bit): Congestion Window Reduced (CWR) flag is set by the sending host to indicate that it received a TCP segment with the ECE flag set and had responded in congestion control mechanism (added to header by RFC 3168).
- **ECE** (1 bit): ECN-Echo has a dual role, depending on the value of the SYN flag:
 - If the SYN flag is set (1), that the TCP peer is ECN capable.
 - If the SYN flag is clear (0), that a packet with Congestion Experienced flag set (ECN=11) in IP header was received during normal transmission (added to header by RFC 3168). This serves as an indication of network congestion (or impending congestion) to the TCP sender.

(*) O "NS" ainda é experimental e não aparece em algumas representações

ECN - Explicit Congestion Notification flags



(*) O "NS" ainda é experimental e não aparece em algumas implementações

Capítulo 3: Sumário

❑ Princípios associados aos serviços da camada de transporte:

- Multiplexação /
desmultiplexação
- Transferência confiável de dados
- Controle de fluxo
- Controle de congestionamento

❑ Instanciação e implementação na Internet.

- UDP
- TCP

Próximo capítulo:

- ❑ Saímos da “borda” da rede (camadas de aplicação e transporte).
- ❑ Entraremos no “núcleo” da rede.